

IOT Based Gas Cylinder Level and Leakage Detection

Abstract:

This project presents an IOT-based Gas Cylinder Level and Leakage Detection system designed to improve safety and efficiency in gas usage. The system employs an ultrasonic sensor to monitor the gas level in the cylinder and a gas sensor to detect any leakage. A Node MCU microcontroller processes the sensor data, displays it on an LCD screen, and transmits it to a cloud platform for remote access. In the event of a gas leak, a buzzer is activated to alert users immediately. This real-time monitoring system enhances user safety by ensuring timely detection of gas depletion and leakage risks.

Keywords: Gas Leakage Detection, Remote Monitoring, Cylinder Safety, Real-Time Alerts.

CHAPTER 1

INTRODUCTION

In recent years, the use of liquefied petroleum gas (LPG) cylinders in residential and industrial applications has become increasingly prevalent due to their efficiency and portability. However, with this rise in usage comes a growing concern regarding the safety, management, and monitoring of gas levels and potential leakage hazards. Gas leaks not only pose serious fire and explosion risks but also contribute to health problems and property damage. Traditional methods of checking gas levels, such as manually lifting the cylinder or relying on rough estimation, are inaccurate, inefficient, and potentially unsafe. Additionally, gas leakage often goes undetected until it escalates into a critical threat. To overcome these limitations, the integration of Internet of Things (IOT) technology provides a smart and reliable solution for real-time monitoring and safety enhancement. The "IOT Based Gas Cylinder Level and Leakage Detection System" is designed to offer a comprehensive and automated approach to gas management. Utilizing sensors and microcontrollers, this system continuously monitors both the gas level and the presence of any hazardous leakage. It provides users with immediate alerts in case of leaks and updates regarding gas availability, ensuring proactive responses before emergencies arise. At the core of this system is the Node MCU microcontroller, a Wi-Fi-enabled platform that not only processes sensor inputs but also connects the system to cloud-based services. This allows for seamless data transmission and real-time access to information through mobile or web interfaces, no matter where the user is located. The ultrasonic sensor calculates the remaining gas by measuring the height of the gas inside the cylinder, while the MQ-series gas sensor detects even minimal leakage levels. The data is displayed on an LCD screen for local monitoring and sent to a remote server for broader accessibility. Additionally, an integrated buzzer is triggered instantly when a gas leak is detected, providing an auditory alert to nearby individuals.

This system not only adds convenience by allowing users to monitor gas usage remotely but also significantly enhances safety by detecting leakage early. It is particularly valuable in homes, restaurants, factories, and other settings where gas is heavily used. The scalable design can be implemented across multiple cylinders or facilities, making it ideal for both individual and commercial use. Overall, the project contributes to smarter energy management, promotes responsible usage, and most importantly, safeguards lives and property.

CHAPTER 2

LITERATURE SURVAY

"Smart LPG Monitoring and Leakage Detection System" **Ramesh K., Priya M. – 2018**

This project involved the development of a microcontroller-based LPG gas monitoring system that aimed to detect leaks and monitor cylinder usage. It used an MQ-6 gas sensor for detecting LPG and alerted the users through a buzzer and SMS via GSM module. The system was built on an Arduino platform and was designed for real-time notification to prevent fire hazards. However, it lacked integration with cloud services or IOT protocols, limiting its remote monitoring capability. Its primary focus was on safety alarms and local alerts, offering a basic, low-cost solution for households.

"LPG Gas Leakage and Level Detection System using Arduino and GSM" **Kumar S., Deepa T. – 2020**

This research proposed a dual-purpose system using Arduino Uno for monitoring gas levels and detecting leakage. It employed an ultrasonic sensor to determine gas level based on distance and MQ-2 sensor for detecting leakage. A GSM module was used to send SMS alerts to users when leakage was detected or gas was running low. The project improved safety in households and commercial kitchens but lacked a cloud database for historical monitoring. It served as a foundational prototype for integrating safety and gas consumption tracking features.

"Smart Gas Cylinder Monitoring System Using IOT" **Bhavana G., Naresh Y. – 2021**

This project introduced a comprehensive gas monitoring system using IOT tools, such as ThingSpeak and Blynk, to provide remote access to cylinder data. It used an ultrasonic sensor to measure gas level and MQ-135 for gas leakage detection. Alerts were provided via mobile notifications and audible alarms. The use of Wi-Fi-based microcontroller (ESP8266) made the setup compact and cost-effective. Though efficient in real-time monitoring, the system lacked scalability for industrial use and focused more on domestic implementations.

"An Intelligent LPG Cylinder Monitoring and Booking System"

Farooq S., Nikhil K. – 2017

This system focused on monitoring LPG gas level and automating the booking process once the gas reached a threshold. A load cell measured the cylinder weight and sent data via GSM. The project emphasized automating gas refills, integrating consumer convenience with safety. However, its reliance on GSM modules made it slower and costlier for long-term use compared to Wi-Fi-based IOT systems. Still, it highlighted the potential for merging commercial services with safety alerts in gas management.

"Gas Leakage Detection with Cloud-Based Alert System"

Sneha R., Karthik M. – 2022

This recent work focused on a cloud-integrated leakage detection system using IOT sensors and platforms like Firebase. It provided not only real-time alerts but also historical data logging for safety audits. An MQ-6 gas sensor and Node MCU handled detection and data transmission. The mobile notifications and web dashboard improved accessibility. Despite its robust features, the system required constant internet connectivity, which might limit rural deployment. However, it showed strong potential for smart home integration.

"IOT-Based Multi-Cylinder Gas Level Monitoring System"

Ajay V., Meghana D. – 2021

This project extended traditional monitoring systems by enabling multiple cylinder tracking in industrial kitchens. It used ultrasonic sensors and a centralized IOT dashboard to display the gas levels of multiple tanks simultaneously. Alerts were generated when any tank approached depletion or leakage was detected using gas sensors. Though cost-effective, synchronization issues among sensors posed challenges. Still, the system demonstrated good scalability and was suitable for commercial kitchens, hotels, and small-scale industries.

"Real-Time LPG Gas Monitoring and Leakage Alert System Using Blynk and Node MCU"

Ashok R., Hema P. – 2020

This study proposed a smart alert system using the Blynk platform for live notifications. The MQ-6 sensor was used for leak detection while ultrasonic sensors tracked gas levels. A Node MCU microcontroller connected to Wi-Fi sent data to the Blynk app, where users could visualize gas levels. Audible buzzers were also included for immediate alerts. Though suitable for domestic use, the design could benefit from stronger enclosure and environmental shielding to ensure accurate sensor readings over time.

CHAPTER 3

IMPLEMENTATION

Existing Systems

Current gas cylinder monitoring systems often rely on manual checks, which can be time-consuming and prone to human error. Automated systems using IOT technology can significantly improve safety and efficiency by providing real-time data on gas levels and detecting leaks early.

Proposed System

The proposed system leverages IOT technology to create a smart and reliable gas monitoring solution. It integrates an ultrasonic sensor to continuously measure the gas level and a gas sensor to detect potential leaks. The Node MCU microcontroller, equipped with a built-in ESP8266 Wi-Fi module, handles data processing and uploads sensor readings to a cloud server for remote monitoring. An LCD provides on-site data display, while a buzzer sounds an alert in the event of a gas leak. This system significantly enhances safety and convenience, reducing the likelihood of accidents and ensuring timely refills.

Block Diagram:

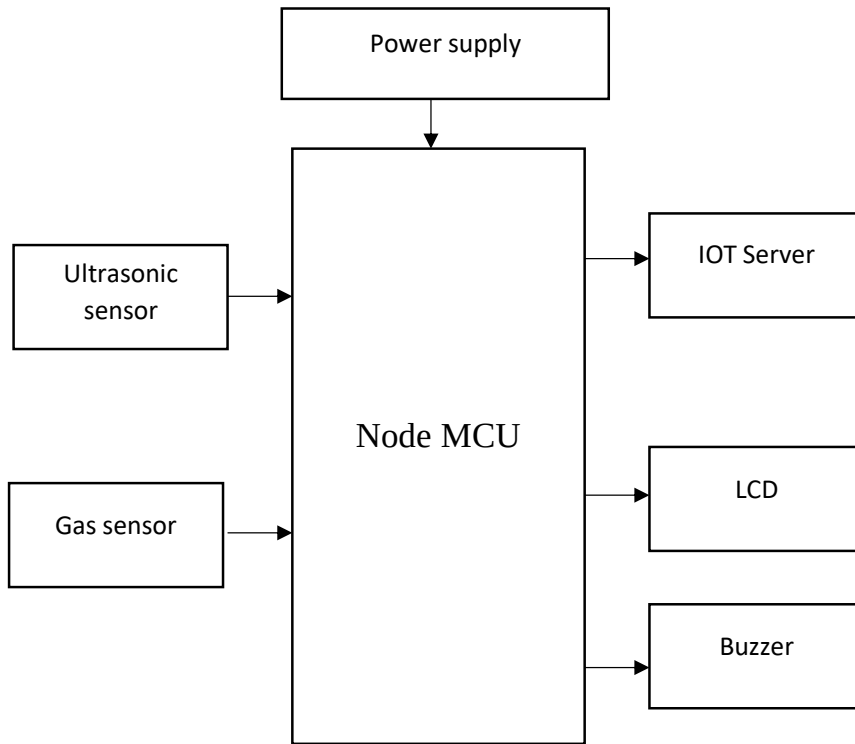


Fig: block diagram for proposed system

Schematic diagram:

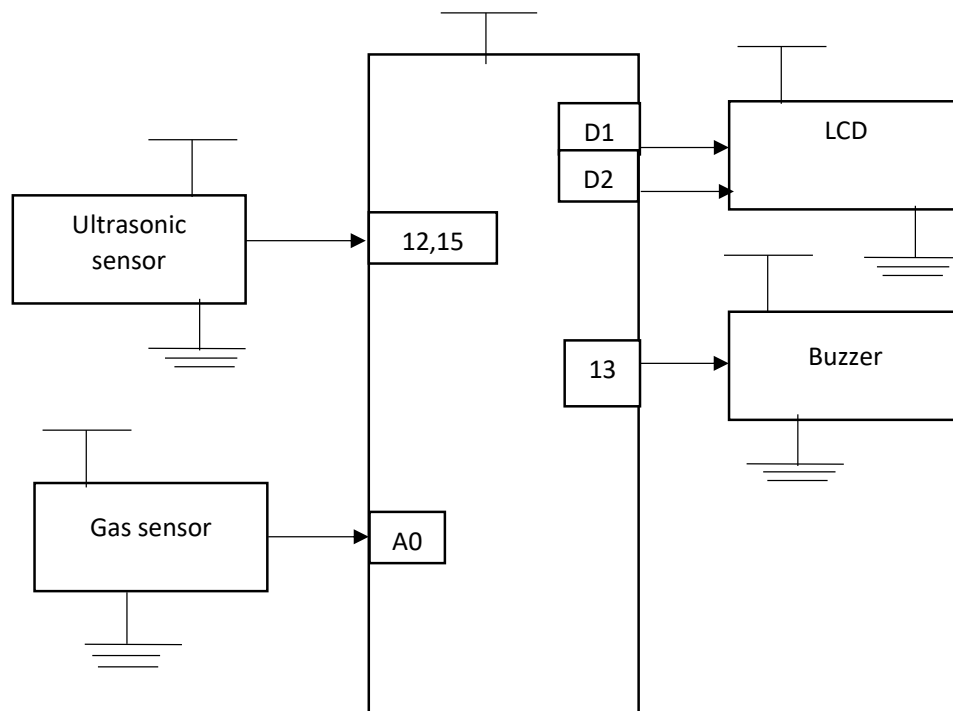


Fig: schematic diagram of proposed system

CHAPTER 4

EMBEDDED SYSTEMS

INTRODUCTION TO EMBEDDED SYSTEMS

The reliance on "embedded systems" in digital technology is increasing on a daily basis. Over ninety-eight percent of the world's CPUs are located in things that the average person would never think of as desktop computers. "Embedded" implies that a computer is "dedicated" to the gadget or platform it runs on. Embedded systems are purpose-built to execute a limited range of predetermined tasks, in contrast to general-purpose computers such as personal computers. Engineers may refine the mechanism for its intended use, which might reduce the product's cost and environmental effect. Companies may take advantage of advantages of scale when they mass-produce embedded systems.

The increasing usage of PC hardware is one of the many notable advances in high-end embedded technologies in the last several years. This shift has made high-end systems more accessible, allowing for initiatives that would have been impossible to fund with the exorbitant cost of non-PC-based embedded technology in the past. The PC platform has nice hardware, but there aren't any good software options. The software in an embedded system is usually stored in read-only memory (ROM) on a single CPU board.

An embedded system is used by almost all electrical devices that have a digital interface. Some examples of such devices include microwaves, watches, video recorders, and automobiles. Despite often having their own operating system, the reasoning within numerous embedded systems is so specialized that it just takes one line of code to write. Embedded systems range in size from small electronic wristwatch that sync to MP3 players to large infrastructure required to operate traffic lights, factories, and sometimes nuclear power stations. The term "firmware" is utilized on occasion to describe the computer programs installed on these CPUs.

Hardware, such as a custom-built computer circuit or an integrated circuit designed specifically for an application (ASIC). By transmitting information from various sensors and detectors to actuators, it is possible to regulate the flow of fuel to an engine or control the functioning of a machine. In an embedded system, software and hardware collaborate.

WHAT IS AN EMBEDDED SYSTEM?

We claim that a system includes embedded capability when its software is built to communicate with its hardware components throughout development. Variation and usefulness are made possible via programming.

Memory, processing units, application-specific integrated circuits, and other similar components are examples of hardware. "Embedded system" may imply whatever you want it to signify, and all of those meanings can be merged into one. Computers with a specific, limited function are known as embedded systems.

SYSTEMS THAT ARE EMBEDDED

Embedded systems have many different kinds of uses. The once-pricing industrial control applications, as well as the automotive, communication, and office appliance industries, as well as the television industry, have recently begun to employ specialist processors. The low cost of embedded systems has led to their widespread use in consumer electronics. The primary goal of most embedded system designers is to facilitate mass manufacturing.

1. The very earliest teller machines
2. Talking over the phone and using cellular networks
3. Hardware components and network interface cards
4. Printers for computers
5. Hard discs
6. Items for automating the house
7. Compact mathematical devices
8. Household appliances
9. instruments used in medicine
10. Measurement instruments
11. Watches that can do many tasks
12. Multi-Use Printers

EMBEDED SYSTEMS HISTORY

Among the first known examples of an orbital guidance computer, created by “Charlie Stark Draper” of the Massachusetts Institute of Technology Measurement Laboratory. The most important component of the project, the Apollo guidance computer, used integrated circuits and newly developed monolithic circuits to decrease size and weight. One of the earliest mass-produced embedded devices was the Autonetics the defendant-17 navigation processor, which was used in the 1961 Minuteman missile launch. Its main memory was kept on a hard drive, and it was built using transistor logic.

In 1966, at the same time as construction of the Minuteman II was underway, an upgraded computer that heavily used integrated circuits supplanted the D-17. This effort was instrumental in reducing the price of quad & gate semiconductors from \$1,000 to \$3, which allowed them to be included in mass-produced devices. Among the first known examples of an orbital guidance computer, created by “Charlie Stark Draper” of the Massachusetts Institute of Technology Measurement Laboratory.

The most important component of the project, the Apollo guidance computer, used integrated circuits and newly developed monolithic circuits to decrease size and weight. One of the earliest mass-produced embedded devices was the Autonetics the defendant-17 navigation processor, which was used in the 1961 Minuteman missile launch. Its main memory was kept on a hard drive, and it was built using transistor logic. In 1966, at the same time as construction of the Minuteman II was underway, an upgraded computer that heavily used integrated circuits supplanted the D-17. This effort was instrumental in reducing the price of quad & gate semiconductors from \$1,000 to \$3, which allowed them to be included in mass-produced devices.

EMBEDED SYSTEMS FEATURES

Embedded computer systems provide a high level of adaptability, allowing them to streamline many processes ranging from product creation to manufacturing. Consequently, they have the potential to be beneficial in any company setting. Specialized operating systems in embedded devices are crucial for intricate, functionality-packed systems. An embedded operating system is a lightweight system that can replicate the functionality of a real-time operating system.

There are instances when embedded systems lack the same level of power or complexity as general-purpose systems. In embedded systems, it is customary to use a CPU with reduced

RAM capacity and low power consumption. Embedded devices often use compact operating systems, despite their limited capability.

Design engineers may enhance the performance of an embedded system by minimizing its physical size while simultaneously improving its effectiveness, speed, and longevity, as it is used only for its designated purpose. A widely adopted approach is to manufacture sure embedded systems in large quantities to take advantage of economies of scale. Regrettably, certain embedded systems are immune to environmental hazards. High humidity and extreme heat are only two instances. Quiet operation is advantageous for devices that possess substantial computing power, such as mobile phones and mobile music players.

Cost is a significant factor to consider in design. Typically, engineers choose for hardware that meets the required capabilities adequately. By implementing an operating system that is real-time or restricting the available programs on general-purpose hardware, it becomes possible for low-volume and trial integrated systems to operate.

EMBEDDED SYSTEM CHARACTERISTICS

Central processing units (CPUs) are often built into embedded computing systems because of the complex features they generally include. Their many non-functional requirements, however, make the work much more difficult.

- Timely completion is essential to avoid system failure
- Multiple rates of operation
- Minimal power usage in a number of cases
- Reducing code size leads to cheaper manufacture.

Functions are often given precedence by programmers focusing on workstations. Even while they may test out sure processing kernels inside their application, they hardly never test out the product's overall performance. The production cost and power use are almost never taken into account. Embedded systems designers need a strong grasp of computer architecture due to the complexity of the programming required to fulfil all of these requirements.

A SYSTEM OF INTEGRATED COMPONENTS

In any system with embedded functionality, you may find a CPU (central processing unit) and other purpose-built hardware. The memory chips that hold the code are part of this hardware.

The term "firmware" may refer to either the software or the actual contents of memory chips. An embedded system's architecture may be conceptualized using a layered framework, displayed in Figure 1.

Any device, such as a desktop machine, laptop, or tablet, may use this layout. The difference between the two must be clearly defined, however. Every embedded system doesn't need its own operating system. Toys, cooling systems, remote controls, and other tiny devices don't need an operating system when their software is developed.

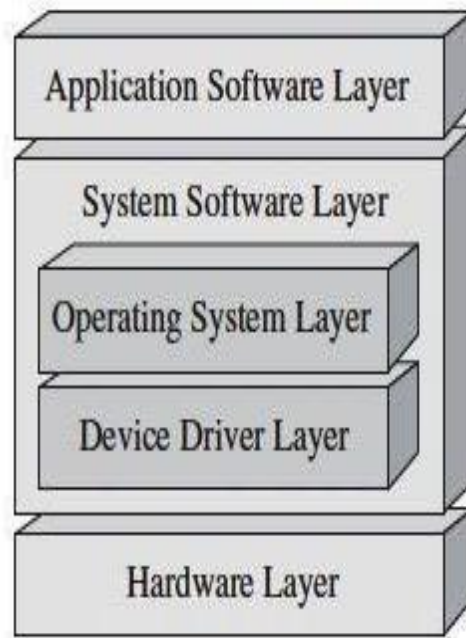


Fig 1.1 Embedded System Layered Architecture

It is recommended to use an operating system while running applications that need sophisticated processing. Inserting the program into the memory chip follows the integration with the operating system. It is not possible to remove software from the memory of a machine chip after it has been installed

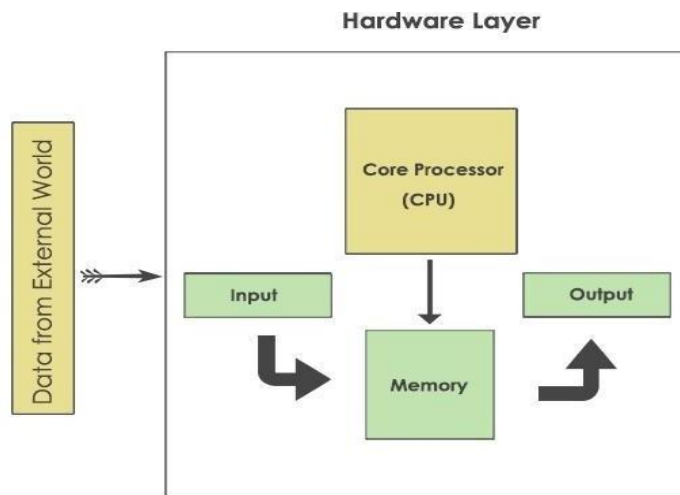


Fig 1.2 Diagram of Embedded systems

Let's take a look at an embedded system's hardware and see how it works. There is no deviation from the figure at all. Core members of the team

Devices for input and output, memory (both random access and read-only), a means of communication, and circuits tailored to certain applications.

A CPU, OR CENTRAL PROCESSING UNIT,

A variety of choices are available, including processing of digital signals, microcontrollers, and microprocessors. The DSP serves as the brains of the operation, processing digital signals. Micro controllers are small and inexpensive computer chips. One of the main selling points is that a single chip will include memory, a digital to analog converter, a series of communications interface, and a plethora of other features. It is appropriate for tiny applications to use a micro-controller since it does not need many other components. However, microprocessors cannot function without a plethora of auxiliary components. Many signal processing applications rely on DSP, including those dealing with audio and video.

Memory

The two most common types of computer memory are RAM (random access memory) and read-only memory (ROM). If a device loses power, all data stored in memory will be wiped. However, data that can only be read will remain untouched. The result is that the ROM has the firmware. The program is loaded into the central processor unit's (CPU) read-only memory (ROM) when power is applied.

Devices for entering data

When compared to desktop computers, embedded systems' input devices are much weaker. It is not easy to communicate with the embedded system of the computer since there is no physical keyboard and mouse. Embedded electronics often take the form of a tiny keypad with a handful of buttons. Numeric inputs are the only valid on a keypad. Embedded systems for process control may gather data collected by sensors and gauges and send it as electrical impulses instead of needing a person to enter data.

Devices for producing results

The output devices of embedded systems also have limited functionality. To display the status of individual components' operations or to indicate the existence of an alarm, embedded systems may use light-emitting diodes (LEDs). Alternatively, a small LCD (liquid crystal display) might indicate certain parameters.

Interfaces for communication

It is not impossible for embedded devices to communicate with one another or for data to be sent to a desktop computer. One way embedded systems make this possible is by including one or more communication interfaces, including USB, IEEE1394, Ethernet, RS422, RS485, or RS232.

DEDICATED CIRCUITRY FOR A PARTICULAR APPLICATION

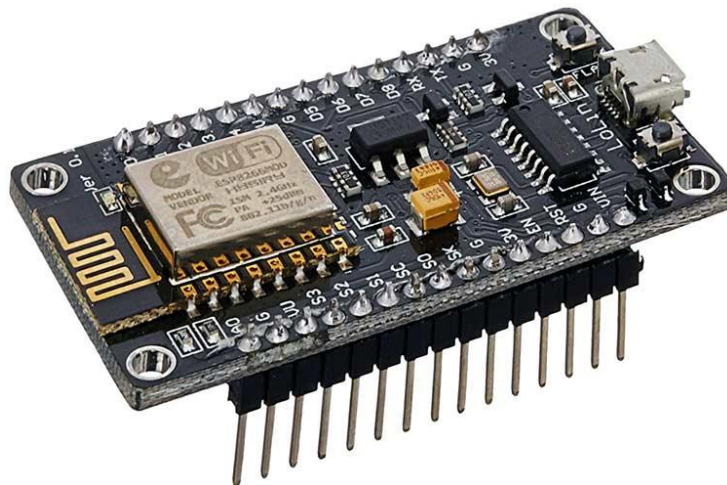
Embedded systems may be equipped with transducers, sensors, and specialized processing and oversight circuits to tackle various tasks. The only way that this circuit and the CPU to communicate is for the CPU to do the essential tasks. All of the machinery needs either a 230-volt outlet or a battery pack to keep running. When making hardware, it's best to aim for the lowest possible power usage.

CHAPTER 5

HARDWARE REQUIREMENTS

Node MCU

Technically speaking Node MCU is a firmware for ESP8266 developed using C Programming Language, Espressif NON-OS SDK and Lua scripting language. Traditionally, we write code for our Microcontrollers like Arduino, STM32, 8051 etc., either in C or C++ and compile it with a set of tools and generate a binary file. This binary file is then uploaded into the flash memory of the microcontroller and it gets executed. Things are quite different with Node MCU. You can consider the Node MCU firmware as an interpreter for Lua Scripts. So, if your ESP8266 is loaded with Node MCU Firmware, you can simply write your application in Lua and send it to the ESP8266. Node MCU Firmware will interpret the byte code and executes the commands. There is no compilation, no binary file etc. Just write a script and run it the team which developed Node MCU Firmware also developed a breakout board for ESP-12E module called the Node MCU Devkit. So, many of us are actually using the board called Node MCU and programming it with Arduino IDE and not the Lua Scripts.



IMPORTANT NOTE: Only one firmware can exist on the ESP8266. It can be either AT Commands Firmware, Node MCU Firmware or Arduino based code. Once you upload an Arduino sketch, the Node MCU firmware gets erased. If you want to work with Lua Scripts and Node MCU, then you have to flash the Node MCU Firmware.

ESP-01 vs. Node MCU (ESP-12E)

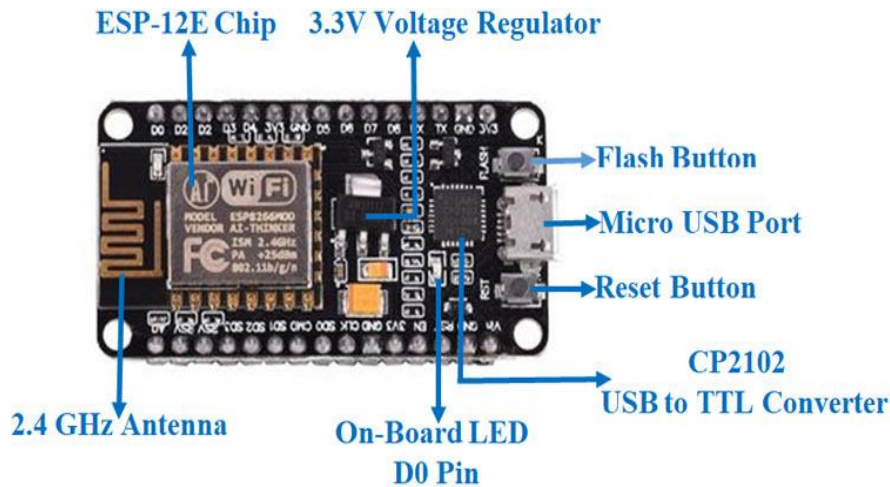
As mentioned earlier, the Node MCU Devkit is actually a Breakout Board for the ESP-12E Module. The ESP-01 is the vanilla version of the ESP8266 Wi-Fi SoC made by Ai-Thinker, a third part module manufacturer for ESP8266.

Brief about Node MCU ESP8266

The **Node MCU ESP8266 development board** comes with the ESP-12E module containing the ESP8266 chip having Tensilica Xtensa 32-bit LX106 RISC microprocessor. This microprocessor supports RTOS and operates at 80MHz to 160 MHz adjustable clock frequency. Node MCU has 128 KB RAM and 4MB of Flash memory to store data and programs. Its high processing power with in-built Wi-Fi / Bluetooth and Deep Sleep Operating features make it ideal for IOT projects.

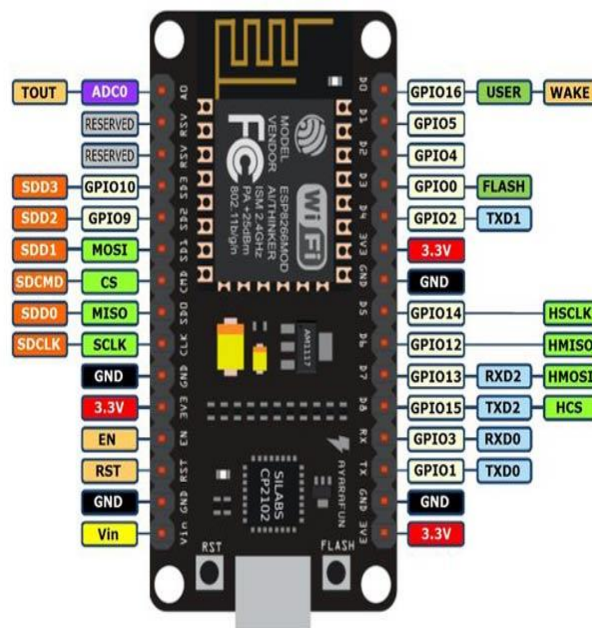
Node MCU can be powered using a Micro USB jack and VIN pin (External Supply Pin). It supports UART, SPI, and I2C interface.

- **ESP8266 Chip:** The core of the Node MCU module is the ESP8266 chip, which provides Wi-Fi connectivity and microcontroller functionality.
- **GPIO Pins:** The Node MCU board includes multiple digital and analog pins that can be used to connect various peripherals such as sensors, LEDs, and motors.
- **Serial Communication Protocols:** It supports UART, SPI, and I2C protocols, enabling communication with other devices and modules.
- **Open-Source Firmware:** The Node MCU firmware is open-source and based on the Lua scripting language, making it easy to program and customize.
- **Compact Design:** The Node MCU module is small and lightweight, making it suitable for various applications, from hobby projects to professional IOT solutions



ESP8266 Wi-Fi module

ESP8266 comes up with powerful processing speed on board. Storage space of this module is also high allowing it to integrate with other devices like sensors. To make this module compatible with other development boards, we have to do level shifting of voltages externally because this board doesn't come up with on-board voltage regulator. This module is cost effective and thus used widely in many applications like the Internet of things and much more. So in this article, we are going to discuss key features of ESP8266, pins specifications, how to use this board for programming purposes and a sample program.



Node MCU ESP8266 Specifications & Features

The Node MCU module is built around the powerful ESP8266 microcontroller, which is well-suited for IOT applications due to its integrated Wi-Fi capabilities and robust processing power. Here's a detailed look at its specifications:

Microcontroller: Tensilica 32-bit RISC CPU Xtensa LX106

The core of the Node MCU is the Tensilica Xtensa LX106, a 32-bit RISC CPU known for its efficiency and performance in handling complex tasks with low power consumption.

Operating Voltage: 3.3V

The module operates at a standard 3.3V, making it compatible with a wide range of sensors and peripherals that also use 3.3V logic levels.

Input Voltage: 7-12V

It accepts an input voltage range of 7-12V, which is regulated down to 3.3V by the onboard regulator, ensuring stable operation and protecting the microcontroller from voltage fluctuations.

Digital I/O Pins (DIO): 16

The Node MCU provides 16 digital input/output pins that can be used to interface with various sensors, actuators, and other digital devices. These pins can be programmed for general purpose use, such as reading sensor data or controlling LEDs.

Analog Input Pin (ADC): 1

There is one analog input pin available, allowing for the connection of analog sensors. This pin can read voltage levels and convert them into digital values for processing.

UARTs: 1

The module includes one UART (Universal Asynchronous Receiver/Transmitter) for serial communication, which is essential for debugging and connecting to other serial devices.

SPIs: 1

One SPI (Serial Peripheral Interface) is available, providing high-speed communication with devices like SD cards, displays, and other SPI peripherals.

I2Cs: 1

The I2C (Inter-Integrated Circuit) bus allows for communication with multiple I2C devices using only two wires, making it ideal for connecting sensors and modules that support I2C communication.

Flash Memory: 4 MB

The Node MCU is equipped with 4 MB of flash memory for storing the firmware and user programs. This ample storage space allows for the development of complex applications and features.

SRAM: 64 KB

It includes 64 KB of SRAM (Static Random Access Memory) for data storage and processing during runtime. This memory is used for variables, data buffers, and program execution.

Clock Speed: 80 MHz

The microcontroller operates at a clock speed of 80 MHz, providing sufficient processing power for handling various tasks efficiently and in real-time.

USB-TTL based on CP2102:

The onboard CP2102 USB-to-TTL converter allows for easy programming and serial communication over USB. It enables plug-and-play functionality, simplifying the setup and development process.

PCB Antenna:

The Node MCU features an integrated PCB antenna, which provides reliable Wi-Fi connectivity without the need for an external antenna. This design keeps the module compact and easy to use.

Compact Size:

The small form factor of the Node MCU module makes it ideal for integration into IOT projects, where space is often limited. Its compact size allows it to fit seamlessly into a wide range of applications, from home automation to wearable devices.

Installation Steps

Now, to install the NODE MCU board in the Arduino IDE, you need to follow the below steps

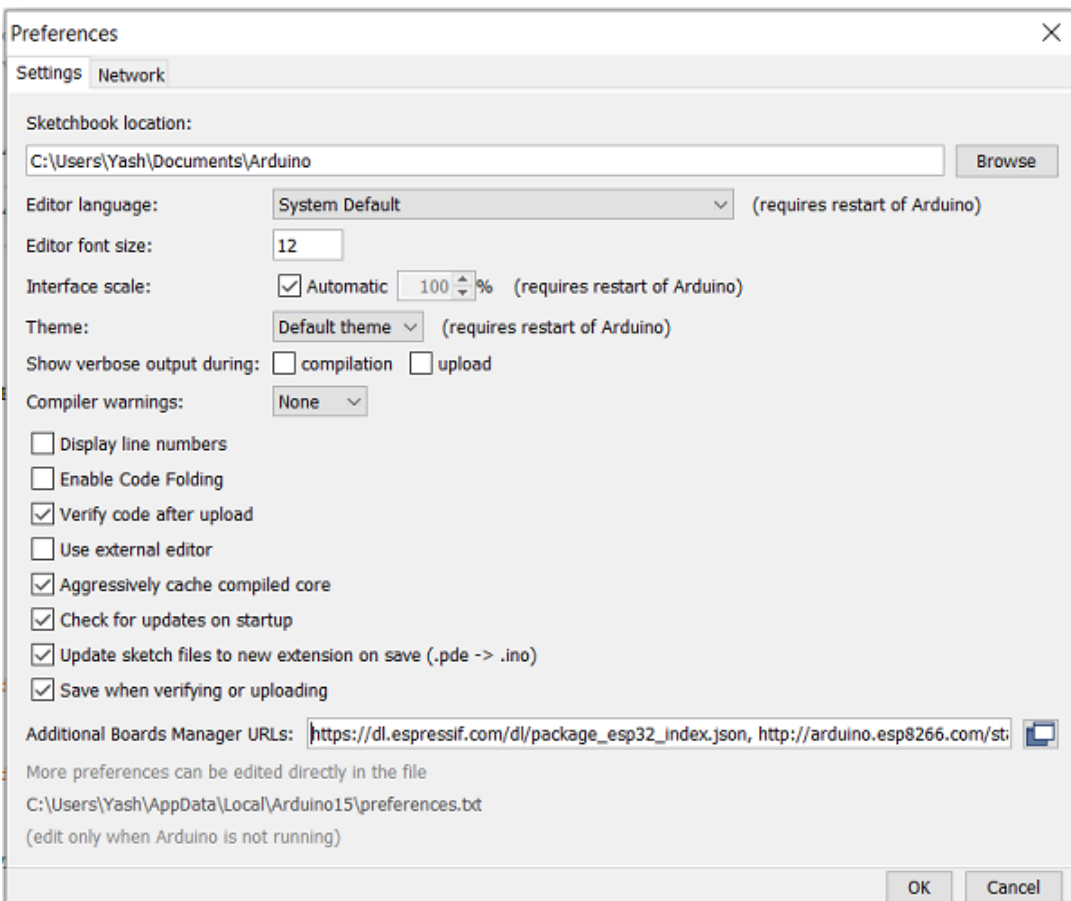
—

- Make sure you have Arduino IDE (preferably the latest version) installed on your machine
- Open Arduino and go to File → Preferences
- In the Additional Boards Manager URL, enter
https://dl.espressif.com/dl/package_esp8266_index.json

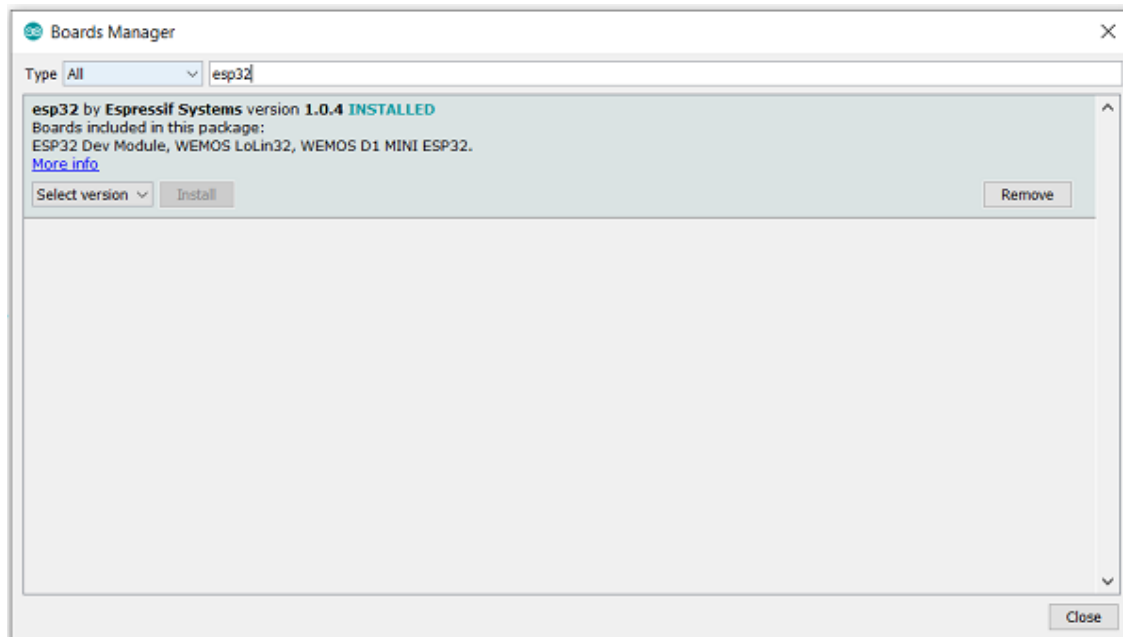
In case you have an existing JSON file's URL in the preferences (this is likely if you've installed ESP8266, stm32duino, or any such additional board in the IDE), you can just append the above path to the existing path, using a comma. An example is shown below, for ESP8266 and NODE MCU boards —

http://arduino.esp8266.com/stable/package_esp8266com_index.json,

https://dl.espressif.com/dl/package_esp8266_index.json

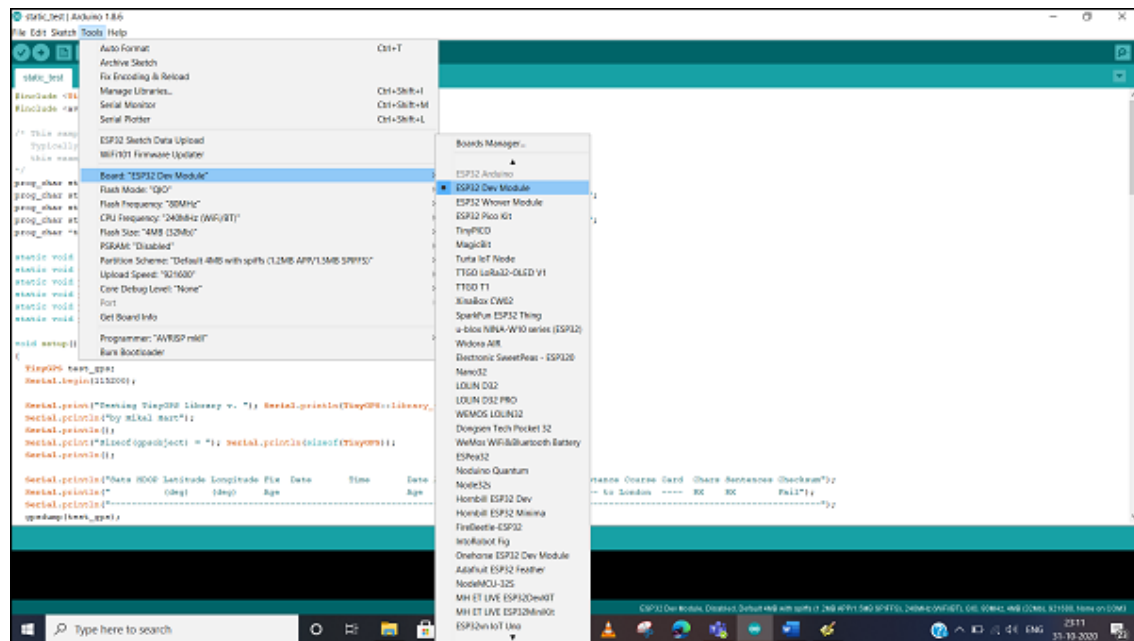


Go to Tools -> Board-> Boards Manager. A pop-up would open up. Search for ESP8266 and install the ESP8266 by **Espressif Systems** board. The image below shows the board already installed because I had installed the board before preparing this tutorial.



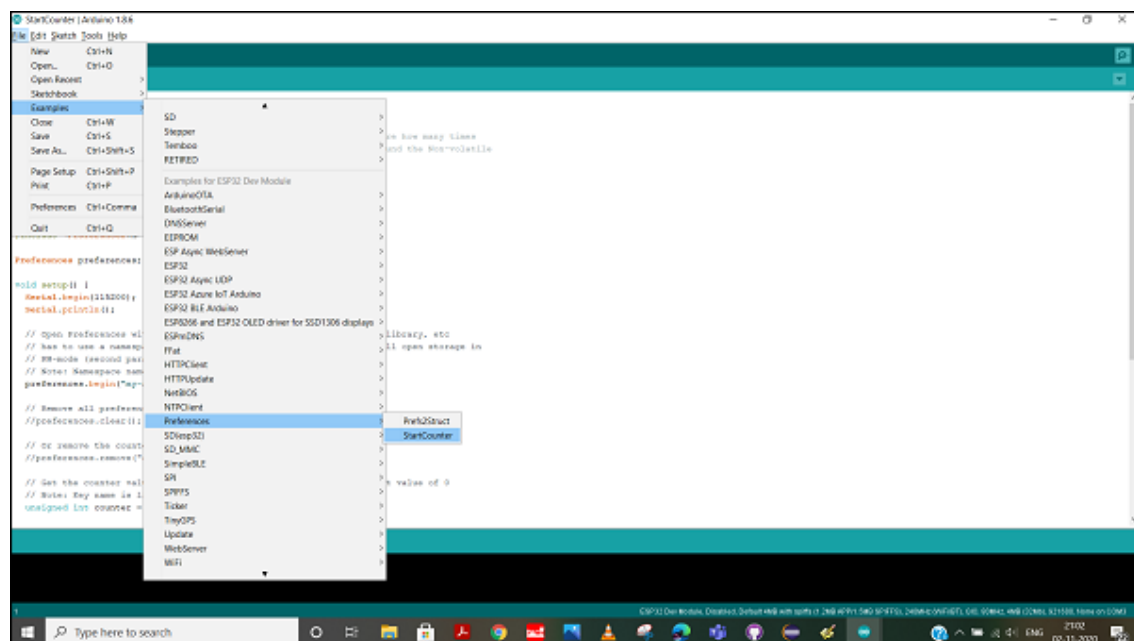
Verifying the Installation

Once your ESP8266 board has been installed, you can verify the installation by going to Tools -> Boards. You can see a whole bunch of boards under the ESP8266 **Arduino** section. Choose the board of your choice. If you are not sure which board best represents the one you have, you can choose ESP8266 **Module**.

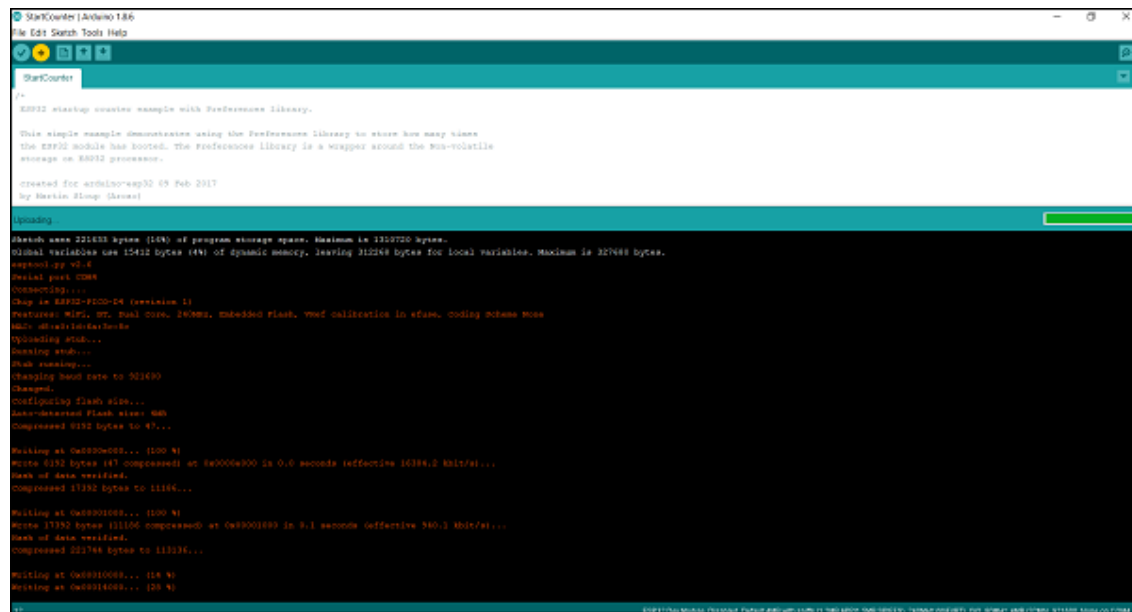


Next, connect your board to your machine using the USB Cable. You should see an additional COM Port under Tools→ Port. Select that additional port. In case you see multiple ports, you can disconnect the USB and see which port disappeared. That port corresponds to ESP8266.

Once the port is identified, pick any one example sketch from File → Examples. We will choose the Start Counter example from File → Examples → Preferences → Start Counter.



Open that sketch, compile it and flash it into the ESP8266 by clicking on the Upload button (the right arrow button, besides the Compile button).



The screenshot shows the Arduino IDE interface. The top toolbar includes buttons for File, Edit, Sketch, Tools, and Help. The main text area displays the sketch code for 'StartCounter', which is an ESP8266 startup counter example using the Preferences library. Below the code, the 'Serial Monitor' window is open, showing the output of the sketch. The output includes memory usage information, a 'Loading' status bar, and a series of status messages indicating the successful upload and execution of the sketch. The status messages include: 'Sketch size 221512 bytes (56% of program storage space. Maximum is 3120720 bytes.', 'Global variables use 15412 bytes (4% of dynamic memory, leaving 312548 bytes for local variables. Maximum is 32768 bytes.', 'Uploading...', 'Uploading done.', 'Sketching board data to 501400 changed.', 'Configuring flash #128...', 'Flash detected flash size: 4096', 'Compressed 5332 bytes to 419...', 'Writing at 0x00000000... (100 %)', 'Write 5132 bytes (47 compressed) at 0x00000000 in 0.0 seconds effective 10184.0 Kbit/s...', 'Flash of data verified.', 'Compressed 17192 bytes to 11184...', 'Writing at 0x00001000... (100 %)', 'Write 17192 bytes (11184 compressed) at 0x00001000 in 0.1 seconds effective 940.1 Kbit/s...', 'Flash of data verified.', 'Compressed 221744 bytes to 111136...', 'Writing at 0x00010000... (100 %)', 'Writing at 0x00014000... (100 %)'.

Then open the Serial Monitor using Tools → Serial Monitor, or simply by pressing Ctrl + Shift + M on your keyboard. You should see the counter value getting incremented after every ESP8266 restart.

Ultrasonic sensor

Ultrasonic sensors are electronic devices that calculate the target's distance by emission of ultrasonic sound waves and convert those waves into electrical signals. The speed of emitted ultrasonic waves traveling speed is faster than the audible sound. There are mainly two essential elements which are the transmitter and receiver. Using the piezoelectric crystals, the transmitter generates sound, and from there it travels to the target and gets back to the receiver component.

To know the distance between the target and the sensor, the sensor calculates the amount of time required for sound emission to travel from transmitter to receiver. The calculation is done as follows:

$$D = 1/2 T * C$$

Where 'T' corresponds to time measured in seconds

‘C’ corresponds to sound speed = 343 measured in mts/sec

Ultrasonic sensor working principle is either similar to sonar or radar which evaluates the target/object attributes by understanding the received echoes from sound/radio waves correspondingly. These sensors produce high-frequency sound waves and analyse the echo which is received from the sensor. The sensors measure the time interval between transmitted and received echoes so that the distance to the target is known.

Ultrasonic Sensor Specifications

Knowing the specifications of an ultrasonic sensor helps in understanding the reliable approximations of distance measurements.

- The sensing range lies between 40 cm to 300 cm.
- The response time is between 50 milliseconds to 200 milliseconds.
- The Beam angle is around 5° .
- It operates within the voltage range of 20 VDC to 30 VDC
- Preciseness is $\pm 5\%$
- The frequency of the ultrasound wave is 120 kHz
- Resolution is 1mm
- The voltage of sensor output is between 0 VDC – 10 VDC
- The ultrasonic sensor weight nearly 150 grams
- Ambient temperature is -25°C to $+70^\circ\text{C}$
- The target dimensions to measure maximum distance is $5\text{ cm} \times 5\text{ cm}$

The ultrasonic sensor pin diagram is:



Ultrasonic Sensor Pin Diagram

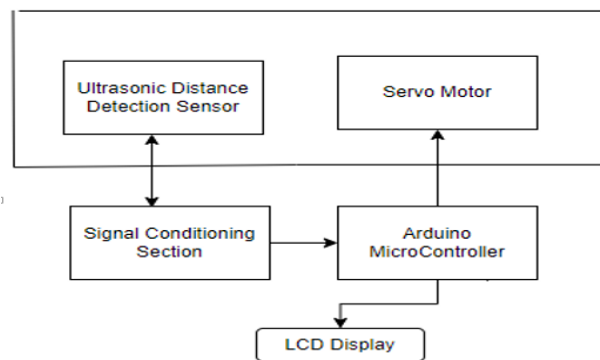
VCC – This pin has to be connected to a power supply +5V.

TRIG – This pin is used to receive controlling signals from the Arduino board. This is the triggering input pin of the sensor

ECHO – This pin is used for sending signals to the Arduino board where the Arduino calculates the pulse duration to know the distance. This pin is the ECHO output of the sensor.

GND – This pin has to be connected to the ground.

The below picture shows the **ultrasonic sensor block diagram** for distance measurement.



Ultrasonic Sensor Block Diagram

The target's distance is calculated using an ultrasonic distance sensor and the output from the sensor is provided to the signal conditioning section and then is processed using an Arduino microcontroller. The results from the microcontroller are fed to the LCD display and then moved to a personal computer.

The ultrasonic sensor can be connected to the servo motor to know the polar distance of the sensor up to 1800 rotations approximately.

Gas sensor

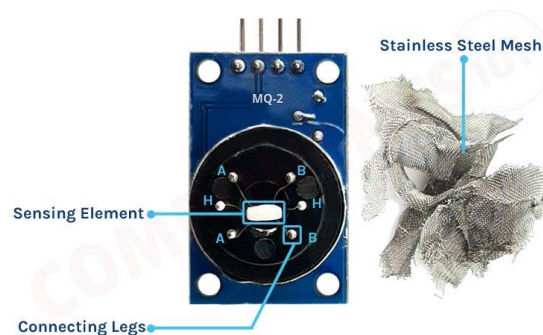
A **gas sensor** is a device which detects the presence or concentration of gases in the atmosphere. Based on the concentration of the gas the sensor produces a corresponding potential difference by changing the resistance of the material inside the sensor, which can be measured as output voltage. Based on this voltage value the type and concentration of the gas can be estimated.



The type of gas the sensor can detect depends on the **sensing material** present inside the sensor. Normally these sensors are available as modules with comparators as shown above. These comparators can be set for a particular threshold value of gas concentration. When the concentration of the gas exceeds this threshold the digital pin goes high. The analog pin can be used to measure the concentration of the gas.

The below image illustrates the parts present in a metal oxide gas sensor

And here is an actual such sensor with its protective cover removed.



The purpose of each of these elements is as below:

Gas sensing layer: It is the main component in the sensor which can be used to sense the variation in the concentration of the gases and generate the change in electrical resistance. The gas sensing layer is basically a chemiresistor which changes its resistance value based on the concentration of a particular gas in the environment. Here the sensing element is made up of a Tin Dioxide (SnO_2) which, in general, has excess electrons (donor element). So whenever toxic gases are being detected the resistance of the element changes and the current flown through it varies which represents the change in concentration of the gases.

Heater coil: The purpose of the heater coil is to burn in the sensing element so that the sensitivity and efficiency of the sensing element increase. It is made of Nickel-Chromium which has a high melting point so that it can stay heated up without getting melted.

Electrode line: As the sensing element produces a very small current when the gas is detected it is more important to maintain the efficiency of carrying those small currents. So, Platinum wires come into play which helps in moving the electrons efficiently.

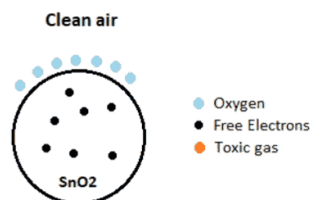
Electrode: It is a junction where the output of the sensing layer is connected to the Electrode line. So that the output current can flow to the required terminal. An electrode here is made of Gold (Au –Aurum) which is a very good conductor.

Tubular ceramic: In between the Heater coil and Gas sensing layer, the tubular ceramic exists which is made of Aluminum oxide (Al_2O_3). As it has a high melting point, it helps in maintaining the burn-in (preheating) of the sensing layer which gives the high sensitivity for the sensing layer to get efficient output current.

Mesh over the sensing element: In order to protect the sensing elements and the setup, a metal mesh is used over it, which is also used to avoid/hold the dust particles entering into the mesh and prevent damaging the gas sensing layer from corrosive particles.

Gas Sensor Working

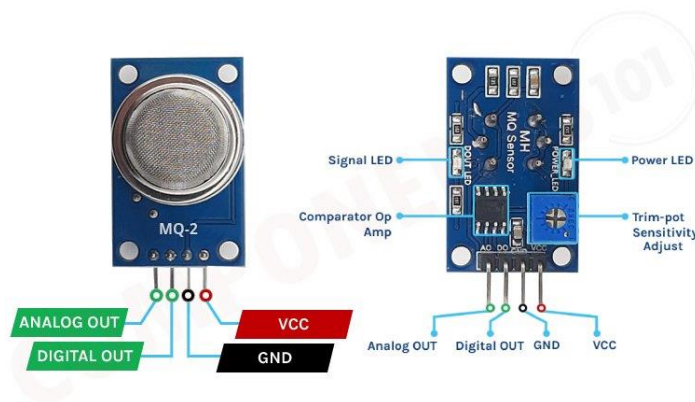
The ability of a Gas sensor to detect gases depends on the **chemiresistor** to conduct current. The most commonly used chemiresistor is Tin Dioxide (SnO_2) which is an n-type semiconductor that has free electrons (also called a donor). Normally the atmosphere will contain more oxygen than combustible gases. The oxygen particles attract the free electrons present in SnO_2 which pushes them to the surface of the SnO_2 . As there are **no free electrons** available output current will be zero. The below gif shows the oxygen molecules (blue color) attracting the free electrons (black color) inside the SnO_2 and preventing it from having free electrons to conduct current.



When the sensor is placed in a toxic or combustible gas environment, the reducing gas (orange color) reacts with the adsorbed oxygen particles and breaks the chemical bond between oxygen and free electrons thus **releasing the free electrons**. As the free electrons are back to their initial position, they can now conduct current, this conduction will be proportional to the number of free electrons available in SnO_2 , if the gas is highly toxic more free electrons will be available.

How to use a Gas sensor?

A basic gas sensor has 6 terminals of which 4 terminals (A, A, B, B) act as input or output and the remaining 2 terminals (H, H) are for heating the coil. Of these 4 terminals, 2 terminals from each side can be used as either input or output (these terminals are reversible as shown in the circuit diagram) and vice versa.



These sensors are normally available as modules (shown right), these modules consist of the gas sensor and a comparator IC. Now let's see the pin description of the gas sensor module which we will generally use with an Arduino. The gas sensor module basically consists of 4 terminals

Vcc – Power supply

GND – Power supply

Digital output – This pin gives an output either in logical high or logical low (0 or 1) that means it displays the presence of any toxic or combustible gases near the sensor.

Analog output – This pin gives an output continuous in voltage which varies based on the concentration of gas that is applied to the gas sensor.

As discussed earlier the output of a gas sensor alone will be very small (in mV) so an external circuit has to be used in order to get a digital high-low output from the sensor. For this purpose, a comparator (LM393), an adjustable potentiometer, and some resistors and capacitors are used.

The purpose of LM393 is to get the output from the sensor, compare it with a reference voltage and display whether the output is logically high or not. Whereas the purpose of the potentiometer is to set the required threshold value of the gas above which the digital output pin should go high.

Most Popular Gas Sensors

Some of the most popular semiconductor gas sensors includes MQ-2, MQ-3, MQ-4, MQ-5, MQ-6, MQ-7, MQ-8, MQ-9, MQ-131, MQ-135, MQ-136, MQ-137, MQ-138, MQ-214, MQ-303A, MQ-306A and MQ-309A. Here are the most important specifications of these sensors.

LCD

LCD is a flat display technology, stands for "Liquid Crystal Display," which is generally used in computer monitors, instrument panels, cell phones, digital cameras, TVs, laptops, tablets, and calculators. It is a thin display device that offers support for large resolutions and better picture quality. The older CRT display technology has replaced by LCDs, and new display technologies like OLEDs have started to replace LCDs. An LCD display is most commonly found with Dell laptop computers and is available as an active-matrix, passive-matrix, or dual-scan display. The picture is an example of an LCD computer monitor.

The Liquid Crystal library allows you to control LCD displays that are compatible with the Hitachi HD44780 driver. There are many of them out there, and you can usually tell them by the 16-pin interface.



Output of the sketch on a 16x2 LCD

The LCDs have a parallel interface, meaning that the microcontroller has to manipulate several interface pins at once to control the display. The interface consists of the following pins:

- A **register select (RS) pin** that controls where in the LCD's memory you're writing data to. You can select either the data register, which holds what goes on the screen, or an instruction register, which is where the LCD's controller looks for instructions on what to do next.
- A **Read/Write (R/W) pin** that selects reading mode or writing mode
- An **Enable pin** that enables writing to the registers
- **8 data pins (D0 -D7)**. The states of these pins (high or low) are the bits that you're writing to a register when you write, or the values you're reading when you read.

There's also a **display contrast pin (Vo)**, **power supply pins (+5V and GND)** and **LED Backlight (Bklt+ and Bklt-)** pins that you can use to power the LCD, control the display contrast, and turn on and off the LED backlight, respectively.

The process of controlling the display involves putting the data that form the image of what you want to display into the data registers, then putting instructions in the instruction register. The Liquid Crystal Library simplifies this for you so you don't need to know the low-level instructions.

The Hitachi-compatible LCDs can be controlled in two modes: 4-bit or 8-bit. The 4-bit mode requires seven I/O pins from the Arduino, while the 8-bit mode requires 11 pins. For displaying text on the screen, you can do most everything in 4-bit mode, so example shows how to control a 16x2 LCD in 4-bit mode.

LCD Pin Description:

The LCD requires 3 control lines (RS, R/W, and EN) and 8(or 4) data lines. The number of data lines depends on the mode of operation. If operated in 8-bit mode then 8-bit data lines are required. And if the operation is in 4-bit mode then 4-bit data lines are required. The 8-bit mode is faster than the 4-bit mode. In 8-bit mode, LCD uses a total of 14 pins including 8 data lines, 3 control lines, and 3 power supply lines (Vee Vss and Vee)

1. **Power Supply:** The LCD discussed here uses three power supply pins (V_{cc} , V_{ss} , and V_{ee}). V_{cc} and V_{ss} pins are used to provide +5V and ground respectively. The pin V_{ee} is used for controlling LCD contrast.
2. **Control Lines:** There are three control lines in the LCD. These three are used to control the LCD operations there are two very important registers inside the LCD: the command register and the data register. The RS (Register select) pin is used to select the register out of these two. If $RS = 0$ the command register is selected and the user is allowed to send the command to the LCD. If $RS = 1$, the data register is selected and the data sent by the user is displayed on the LCD.
 R/W (Read/ Write) pin allows the user to read/write the information (data or code) to/ from the LCD. $R/W = 1$ when reading and $R/W=0$ when the writing operation is performed.
Another control pin EN (Enable) is used to latch the data present on the data pins. A high-low signal is required to latch the data. The LCD interprets and executes the commands at the instant the EN line is brought low.
3. **Data lines:** The 8-bits data pins. D (0)-D (7) are used to send the information to the LCD or read the contents of the LCD's internal register.

LCD Interfacing:

The LCD can be interfaced to the microprocessor 8085 using the programmable peripheral interface (PPI-8255) IC. To display letters and numbers. ASCII code for the letters A to Z, a to z, and numbers 0 to 9 is sent to the data lines (D0 -D7). These codes may be sent to LCD data lines through one port of 8255 (PPI), port A is used as the output port and send the data to the LCD. The EN pin and RS pin are connected to port B of the 8255. Since it is used as a normal display R/W is made low by connecting to the ground directly. Power supply connections are provided to V_{cc} and V_{ss} pins. The V_{ee} pin is connected to the EE moving node of the potentiometer which is connected between the V_{cc} and V_{ss} pins. By moving the potentiometer the contrast of the LCD can be changed.

IOT (Internet of Things)

The Internet of Things (IOT) refers to a network of physical devices, vehicles, appliances, and other physical objects that are embedded with sensors, software, and network connectivity, allowing them to collect and share data.

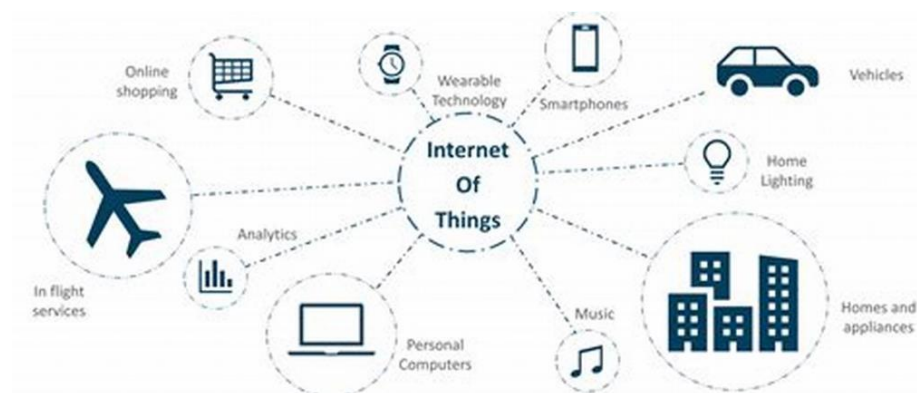
IOT devices—also known as “smart objects”—can range from simple “smart home” devices like smart thermostats, to wearables like smart watches and RFID-enabled clothing, to complex industrial machinery and transportation systems. Technologists are even envisioning entire “smart cities” predicated on IOT technologies.

IOT enables these smart devices to communicate with each other and with other internet-enabled devices. Like smartphones and gateways, creating a vast network of interconnected devices that can exchange data and perform various tasks autonomously. This can include:

- monitoring environmental conditions in farms
- managing traffic patterns with smart cars and other smart automotive devices
- controlling machines and processes in factories
- tracking inventory and shipments in warehouses

The potential applications of IOT are vast and varied, and its impact is already being felt across a wide range of industries, including manufacturing, transportation, healthcare, and agriculture. As the number of internet-connected devices continues to grow, IOT is likely to play an increasingly important role in shaping our world. Transforming the way that we live, work, and interact with each other.

In an enterprise context, IOT devices are used to monitor a wide range of parameters such as temperature, humidity, air quality, energy consumption, and machine performance. This data can be analyzed in real time to identify patterns, trends, and anomalies that can help businesses optimize their operations and improve their bottom line.



Top IOT Platforms

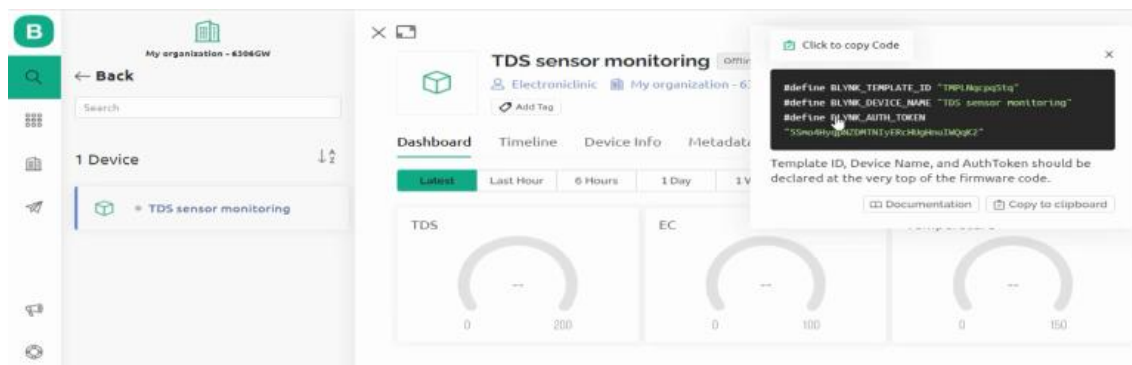
1. Blynk

2. Thing Speak
3. Azure IOT Hub
4. AWS IOT Core
5. Google Cloud IOT
6. IBM Watson IOT Platform
7. Particle
8. Adafruit IO
9. Losant
10. Thingier.io
11. Kaa IOT

Blynk is a popular Internet of Things (IOT) platform that makes it easy to build mobile and web applications for smart devices. The platform provides a user-friendly interface, enabling users to connect their hardware devices to the Blynk Cloud, control them remotely, and visualize data from sensors. Blynk supports a wide range of hardware platforms, including Arduino, Raspberry Pi, ESP8266, and more.

Some of the key features of Blynk include:

- Drag-and-drop interface for creating custom dashboards.
- Real-time monitoring and control of connected devices.
- Support for multiple hardware platforms.
- Integration with various communication protocols, such as Wi-Fi, Bluetooth, and GSM.



Buzzer

An audio signalling device like a beeper or buzzer may be electromechanical or piezoelectric or mechanical type. The main function of this is to convert the signal from audio to sound. Generally, it is powered through DC voltage and used in timers, alarm devices, printers, alarms, computers, etc. Based on the various designs, it can generate different sounds like alarm, music, bell & siren.



Buzzer Pin Configuration

The **pin configuration of the buzzer** is shown below. It includes two pins namely positive and negative. The positive terminal of this is represented with the '+' symbol or a longer terminal. This terminal is powered through 6Volts whereas the negative terminal is represented with the '-' symbol or short terminal and it is connected to the GND terminal.

POWER SUPPLY:

REGULATED POWER SUPPLY

Virtually every component of an electronic system converts DC electricity. Therefore, the DC power source will be necessary for each of these stages. A battery can power any system that uses little electricity. However, batteries may not be the most straightforward or affordable option for gadgets that need constant power. An unregulated power supply, which includes a filter, rectifier, and transformer, is the most effective option. Down below, you can see the diagram.

The electrical circuits inside any given device must be capable of supplying a consistent DC voltage within the device's specified power supply limit. Both the voltage and the current may only go as high as this DC supply allows. The problem is that the electrical devices are vulnerable to breakdowns caused by fluctuations in the mains supply.

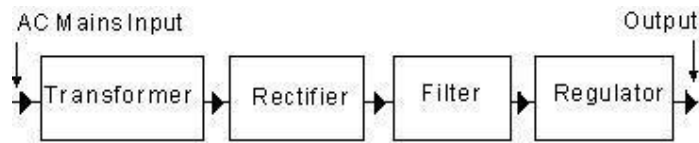


Fig 3.5 Block diagram of Regulated power supply

The DC power supply must reach a certain operating point, sometimes called the Q-point or quiescent point, for all electronic devices, active and passive. To lower the voltage to a level that electronics can handle, a step down transformer is used. A 230 volt 1 Ø supply is accessible in India. After passing through a rectifier, the sinusoidal AC voltage that the transformer produces becomes pulsing DC.

This output is used to reduce AC ripples by sending the DC components via a filter circuit. But there are several downsides to using an uncontrolled power source.

No matter what happens to the mains alternating current (ac) or how much the load fluctuates, a regulated power supply electrical circuit will keep the voltage between the load terminals at a constant direct current (dc).

Simply said, a regulated power supply is an ordinary power supply with a voltage regulator connected to it, as seen in the image. A conventional power supply supplies current to a voltage regulator, which in turn produces the desired output. An output voltage remains constant regardless of changes to either the ac input voltage or the output (or load) current..

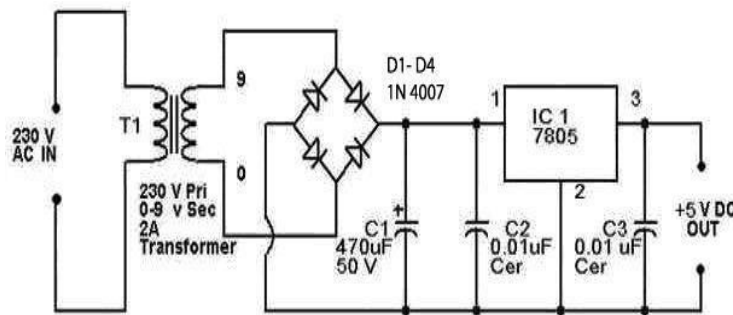


Fig 3.6 Circuit diagram of Regulated power supply

TRANSFORMER

Simply said, a transformer is a static device that aids in the conversion of electrical power from one circuit to another, while maintaining the same frequency. In a circuit, changing the voltage

requires changing the current ratings in a direct proportional fashion.

TRANSFORMER WORKING PRINCIPLE

A transformer operates on the principle of mutual inductance, which occurs when two circuits linked by a common magnetic flux experience a change in current. Two electrically separate coils that are inductively linked are magnetically coupled via a reluctance channel to make a basic transformer. To better grasp the operation of the transformer, refer to the diagram provided below.

Below, you can see the primary and secondary windings of a power transformer. In the core's cross-section, you can make see the little spaces that exist between the lamination strips. Imbricated is the word that best defines these staggered joints. The two coils have a rather large mutual inductance. A mutual electromotive force is induced in the transformer by the coil, which is connected to an alternating voltage source, via the alternating flux that builds up in the laminated core.

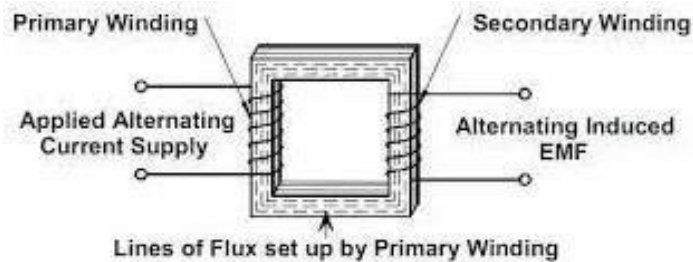


Fig 3.7 Working principle of Transformer

The majority of the alternating current generated by this coil is coupled with the other coil, resulting in the production of the electromotive force that is produced mutually. Using Faraday's equations of electromagnetic induction, we can describe the so-produced electromotive force as

$$e = M \cdot \frac{dI}{dt}$$

- A current runs through the second coil circuit while it is closed, allowing electrical energy to be magnetically transferred from the first coil to the second. The first coil, often known as the main winding, receives the alternating current source. Another name for the second coil, which is where the energy is extracted, is the secondary

winding.

- To summarise, a transformer is responsible for the following tasks:
- The movement of current from one electrical circuit to another.
- The conveyance of electrical energy with no alteration to the frequency.
- Use electromagnetic induction as a means of transfer.
- Two electrical circuits are connected by the principle of mutual induction..

CLASSIFICATION OF TRANSFORMER

- Step-Up Transformer
- Step-Down Transformer

Step-Down Transformer

Typically, a step down transformer will reduce the voltage from a higher level or phase arrangement to a lower one. They may have functions for control and instrumentation, power distribution, and electrical isolation. The conversion of voltage and/or current levels is accomplished by step down transformers by use of the magnetic induction concept between coils. As an example, consider a 2:1 ratio of 100 main turns to 50 secondary turns. Simply said, step down transformers are devices that convert voltages.

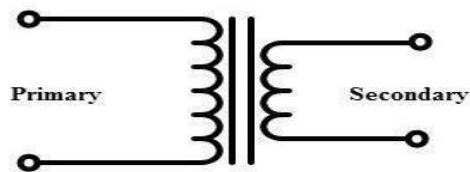


Fig 3.8 Step Down Transformer

Step-Up Transformer

A step-up transformer's secondary coil has more turns of wire, resulting in a higher induced voltage. The transformer is referred to as a step up transformer due to the fact that the voltage output exceeds the input voltage. The design of a step-up transformer is characterised by a secondary voltage that is higher than the primary voltage, in this case 110v 220v. When used with a higher voltage, this transformer "steps up" the current. To operate a 220v appliance in a nation that uses a 110v power source, for example, a step up transformer is required.

Transforming one voltage into another is the job of a step up transformer, which can handle alternating current (AC) of 110v to 220v. It is able to "step-up" or "step-down" voltage and operates on the concept of magnetic induction; it does not have any moving components. Accordingly, voltage may be increased using a step-up transformer and decreased using a step-down transformer.

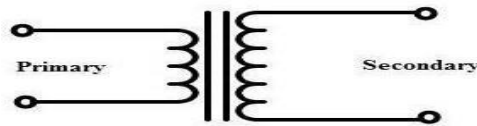


Fig 3.9 Step Up Transformer

FULL WAVE RECTIFIER

If a waveform has both positive and negative cycles, a full wave rectifier will rectify both of them. The average output of this full-wave rectifier is greater than that of a half-wave rectifier, which is one of its advantages. The output has fewer alternating current (AC) components than the input.

There are essentially many subtypes of full wave rectifiers.

- Centre Tapped Full Wave Rectifier
- Full Wave Bridge Rectifier

The bottom half of the transformer will be positively correlated with the top half during the subsequent cycle. Which means that diodes D2 and D3 are forward biased during this cycle. The current travels down route 3-2 before turning around and heading back up path 4-1. Reverse bias is present in diodes D1 and D4. Therefore, pathways 1-2 and 3-4 do not carry any current. This fixes the negative cycle, and it shows up throughout the load.

Components involved in filter Circuit

The inductor and capacitor are the standard components of a filter circuit. A capacitor only lets current flow in one direction, whereas an inductor only lets direct current (DC) through. This means that inductors and capacitors, when combined, may filter the signal effectively for different applications.

CAPACITOR FILTER

A high-value capacitor, positioned across the load resistor, makes up this filter. This capacitor is charged during the conducting phase and then used to provide energy to the load when it is not conducting. By doing so, the ripple component is greatly reduced and the time length that the current travels through the load resistor is increased.

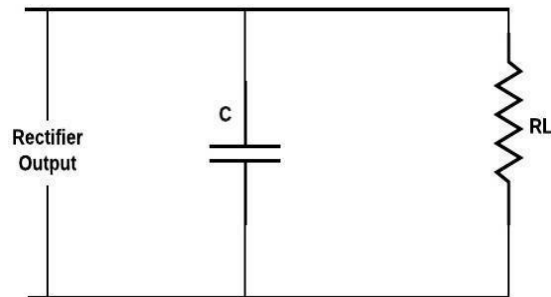


Fig 3.14 Capacitor Filter

The function of the capacitor filter may be viewed in terms of Impedance $\frac{1}{2\pi f + C}$ to the ripple component of frequency f . This ripple component of current, therefore, gets bypassed through C and only the DC component flows through the load resistor R_L .

VOLTAGE REGULATOR

A voltage regulator is a kind of electrical regulator that is specifically designed to automatically keep the voltage level constant. A combination of active and passive electrical components, as well as an electromechanical mechanism, might be used. A number of AC or DC voltages may be controlled by it, depending on its architecture.

The purpose of a voltage regulator is to "regulate" the voltage level automatically. It takes the input voltage and reduces it to the required level, then maintains that level throughout the supply. As a result, the voltage remains constant regardless of the supplied load.

All three of these pins are regulators..

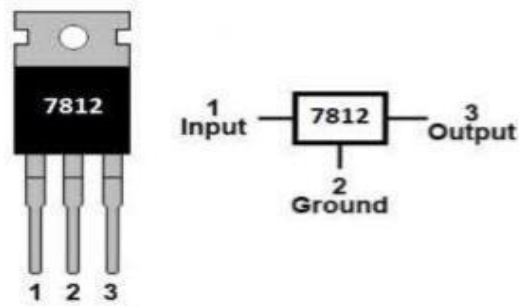


Fig 3.15 12 V Voltage Regulator

There are two types of regulator are they.

- Positive Voltage Series (78xx) and
- Negative Voltage Series (79xx)

Positive Voltage Series (78xx)

A voltage rating of '05' and a positive series of '78' are both shown. There are two voltage regulators—the 7802 and the 7812—that are required for this project.

The 7805 can generate up to 5 volts. A regulator output of 5V is indicated by the number '05'.

And 12V is the maximum output of the 7812. 12V is the output voltage of the regulator.

CHAPTER 6

SOFTWARE CONSTRAINTS

ARDUINO IDE: AN INTRODUCTION

Simple, open-source software and hardware form the basis of Arduino, a platform for prototyping that everyone may utilize. The heart of the Arduino system is a microcontroller, and the legs are the software that writes and uploads programs to the board. The Integrated Development Environment (IDE) for Arduino is the name of the program.

FEATURES OF ARDUINO IDE

Arduino boards can receive signals from a variety of sensors and use them to perform stuff like link to the cloud, turn on/off lights or motors, and even digitally generate signals.

By programming the board's microcontroller via the Arduino IDE, you may manage its operations.

An Arduino, or programmable circuit board does not need a separate tool, a programmer, to install new software. All you need is a USB cord.

To make things even simpler to grasp, the Arduino IDE employs a stripped-down version of C++.

Arduino makes the microcontroller's features more approachable because to its consistent physical size.

Getting a feel for the Arduino UNO board's fundamentals is necessary before installing the software for the Arduino development environment. These details will allow us to program the Arduino board.

DATA TYPES FOR ARDUINO

The "data types" module in C is a useful tool for building various kinds of functions and variables. How much data may be stored in a variable and what its bits mean are both dictated by its type?

Example:

```
Void Loop ( )  
  
{  
  
    // rest of the code  
  
}
```

Boolean

A Boolean can only take on the values true or false. In a code, one byte is used for every Boolean value.

Example:

```
Boolean state= false ; // declaration of variable with type boolean and initialize it  
with false.
```

```
Boolean state = true ; // declaration of variable with type boolean and initialize it  
with false.
```

Char

A type of information that measures exactly one byte and can store the contents of a single character. You must use single quotes around both "ABC" and "A" as the former is a string and the latter is a character literal.

Whatever the case may be, numerical storage is the true technique of characters. The ASCII chart shows the precise encoding that was utilized. Due to this, mathematical operations that include a character may utilize its ASCII value. The ASCII code of the letter initial A is 65, hence adding a single to the length of 'A' gets the number 66.

Example:

Char chr_a = 'a' ;//declaration of variable with type char and initialize it with character a.

Char chr_c = 97 ;//declaration of variable with type char and initialize it with character 97

Unsigned char

Unsigned char is an unsigned data type that occupies one byte of memory. The unsigned char data type encodes numbers from 0 to 255.

Example:

Unsigned Char chr_y = 121 ; // declaration of variable with type Unsigned char and initialize it with character y

Byte

A byte stores an 8-bit unsigned number, from 0 to 255.

Example:

byte m = 25 ;//declaration of variable with type byte and initialize it with 25

Int

Integers are the primary data-type for number storage. `int` stores a 16-bit (2-byte) value. This yields a range of -32,768 to 32,767 (minimum value of -2^{15} and a maximum value of $(2^{15}) - 1$).

The `int` size varies from board to board. On the `Arduino Due`, for example, an `int` stores a 32-bit (4-byte) value. This yields a range of -2,147,483,648 to 2,147,483,647 (minimum value of -2^{31} and a maximum value of $(2^{31}) - 1$).

Example:

```
int counter = 32 ;// declaration of variable with type int and initialize it with 32.
```

Unsigned int

Unsigned `int`'s (unsigned integers) are the same as `int` in the way that they store a 2 byte value. Instead of storing negative numbers, however, they only store positive values, yielding a useful range of 0 to 65,535 ($2^{16} - 1$). The `Due` stores a 4 byte (32-bit) value, ranging from 0 to 4,294,967,295 ($2^{32} - 1$).

Example:

```
Unsigned int counter= 60 ; // declaration of variable with type unsigned int and initialize it with 60.
```

Word

On the Uno and other ATMEGA based boards, a word stores a 16-bit unsigned number. On the Due and Zero, it stores a 32-bit unsigned number.

Example

```
word w = 1000 ;//declaration of variable with type word and initialize it with 1000.
```

Long Long variables are extended size variables for number storage, and store 32 bits (4 bytes), from 2,147,483,648 to 2,147,483,647.

Example:

```
Long velocity= 102346 ;//declaration of variable with type Long and initialize it  
with 102346
```

Unsigned long

Unsigned long variables are extended size variables for number storage and store 32 bits (4 bytes). Unlike standard longs, unsigned longs will not store negative numbers, making their range from 0 to 4,294,967,295 ($2^{32} - 1$).

Example:

```
Unsigned Long velocity = 101006 ;// declaration of variable with type Unsigned Long  
and initialize it with 101006.
```

Short

A short is a 16-bit data-type. On all Arduinos (ATMega and ARM based), a short stores a 16-bit (2-byte) value. This yields a range of -32,768 to 32,767 (minimum value of - 2^{15} and a maximum value of $(2^{15}) - 1$).

Example:

```
short val = 13 ;//declaration of variable with type short and initialize it with 13
```

Float

A decimal number is the data type to floating-point numbers. Due to its higher resolution in comparison to integers, floating-point numbers are frequently employed to approximate the analogue and continuous values.

The maximum and minimum values for a floating-point number are also both $3.4028235E+38$. They take up a total of 32 bits (4 bytes) of storage space.

Example:

```
float num = 1.352; //declaration of variable with type float and initialize it with  
1.352.
```

Double

Double precision floating-point numbers require four bytes on the Uno and other ATMEGA based circuits. In other words, there is no additional accuracy gained by using a double instead of a float. Precision for doubles on the chip known as the Arduino Due is 8 bytes (64 bits).

Example:

```
double num = 45.352 ;// declaration of variable with type double and initialize it with  
45.352.
```

HOW TO UPLOAD THE ARDUINO BOARD PROGRAM

Once you've followed these easy instructions, the Arduino IDE will be running on your computer and the board will be prepared to accept software in a USB connection.

Step 1

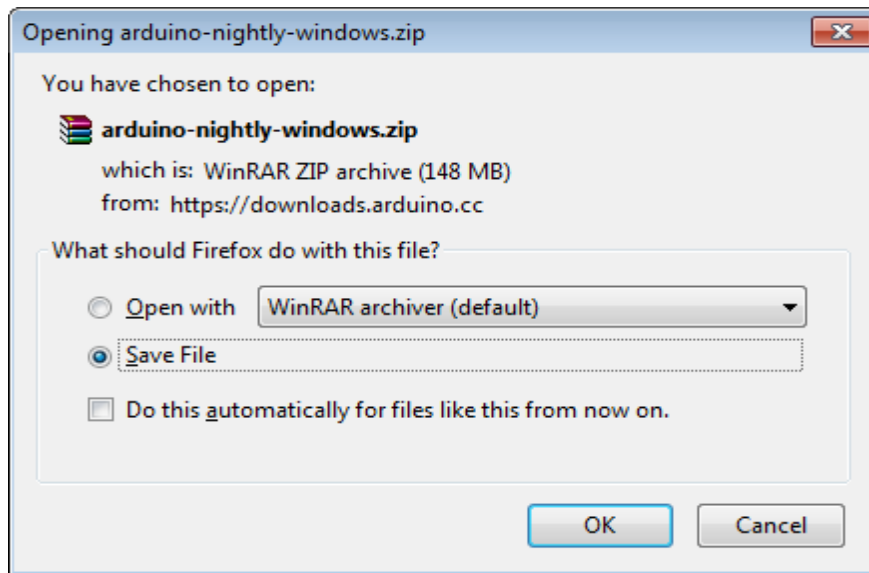
An Arduino board—or any other suitable board—and a USB cable are the primary necessities.

To link a USB printer with an Arduino Uno, a Duemilanove, Mega2560, or Diecimila, you'll need a standard USB cable...



Fig 4.1 USB Cable

Step 2: Arduino IDE software may be downloaded.



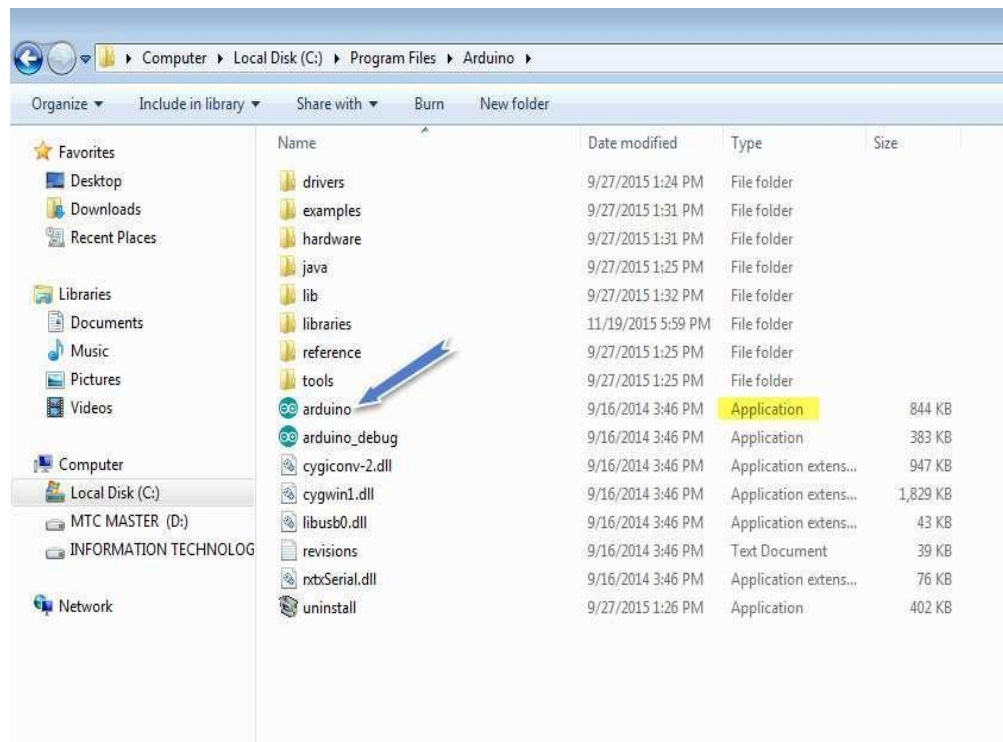
You may get many versions for the Arduino IDE on the Download page of the Arduino organization. Choose an app that is compatible with your OS, whether it iOS, Windows, or Linux. When the download is complete, open the archive.gz file.

Step 3: Activate your board.

The Micro, Duemilanove, Arduino Uno, or Mega systems have the capability to either take power from external sources or establish a connection to a computer's USB port. In order to use the Leonardo Diecimila board, it is necessary to set it to accept power via the USB cord. The user may choose a power source by putting a small plastic device known as a "jumper" in one of all three terminals that connect the HDMI to power connections.

Step 4: Open the Arduino Integrated Development Environment (IDE).

Upon completion of the code download, the first action is to extract the folder that encompasses the Arduino IDE. The eternally named program icon (application.exe) may be found inside this folder. To start the Integrated Development Environment (IDE), just perform a double-click action on the icon.

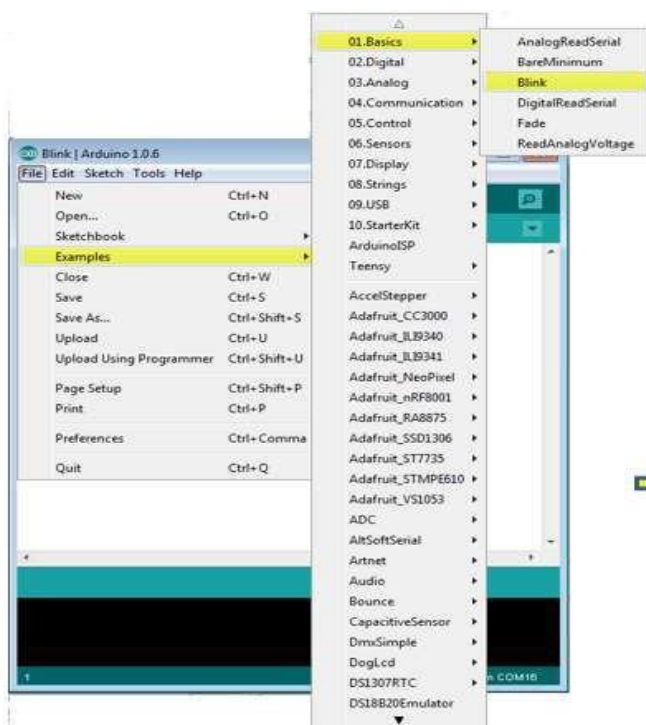
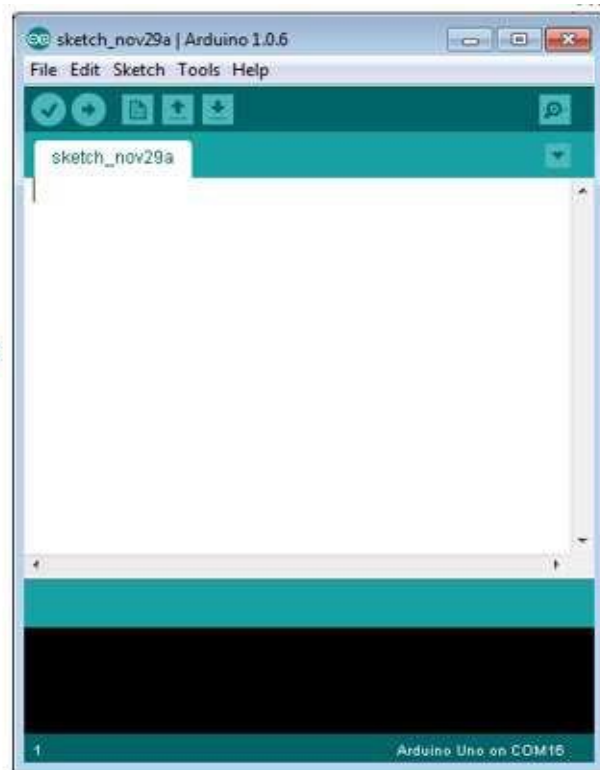
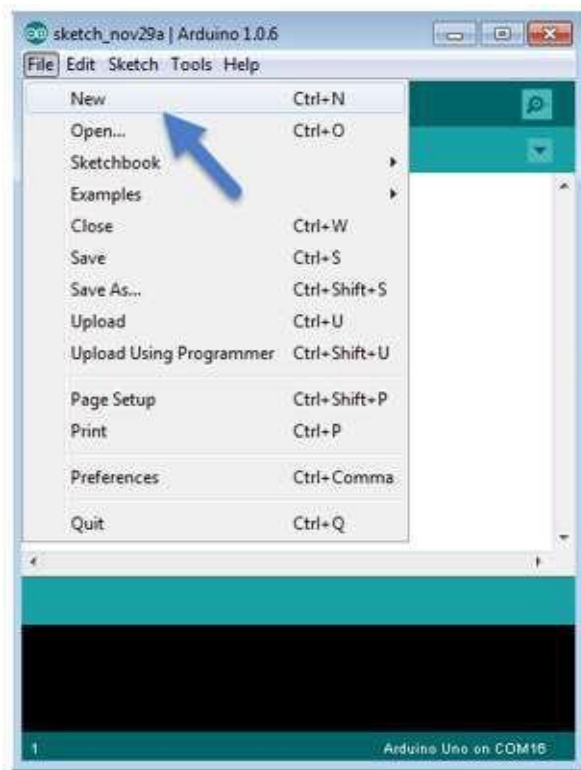


Step 5: Open your first project.

Once the software starts, you have two options: Create a new project.

Open an existing project example.

To create a new project, select File --> New To open

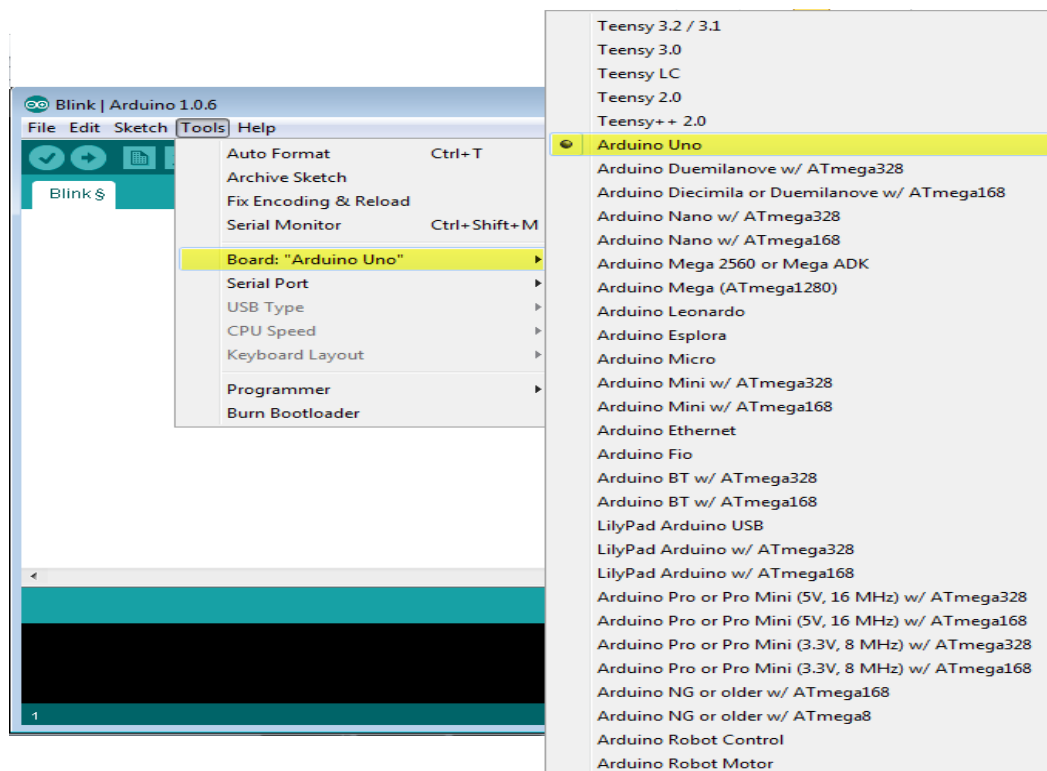


There is no text. By doing this, it delays the LED's cycle of turning on and off. Instead, you might use one of the alternative instances that are accessible.

Step 6: Select your Arduino board.

To avoid any error while uploading your program to the board, you must select the correct Arduino board name, which matches with the board connected to your computer.

Go to Tools → Board and select your board

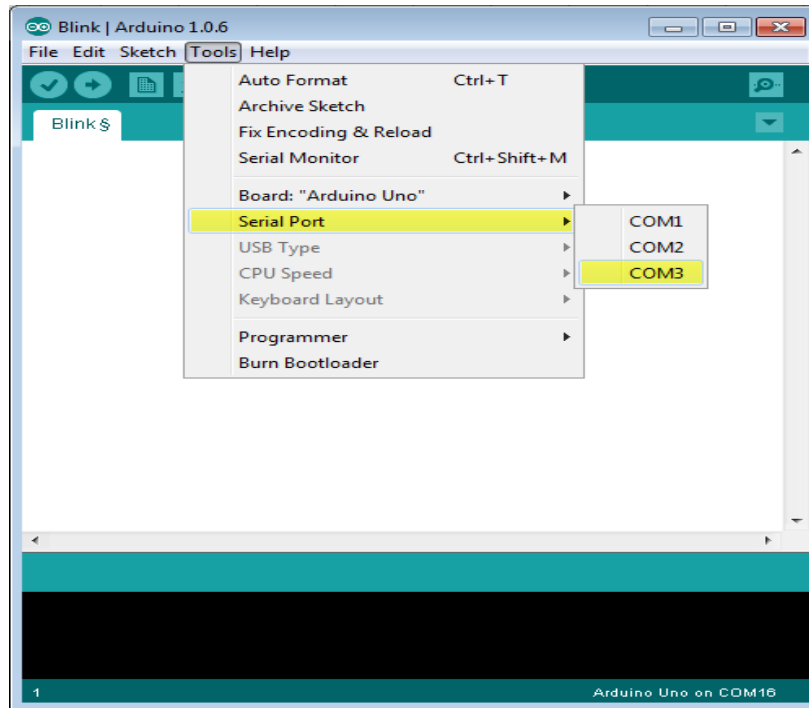


Here, we are selecting a Blink among a vast array of options. In this manner, the activation and deactivation sequence for the LEDs will be postponed. Alternatively, one might use a different of the readily available case studies.

Step 7: Locate the serial connection on your device.

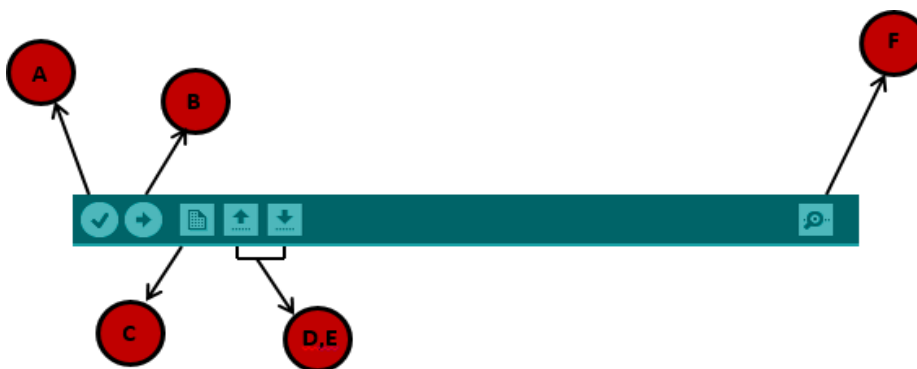
Confirm whether a serial port has been chosen on the board that runs Arduino. To get to the "Tools" menu, choose the option for accessing your UART Port. It is extremely probable this is COM3, because COM1 or COM2 are often used for hardware serial connections. Once you

have uninstalled the Arduino board, we may dismiss the menu and check whether the corresponding item has been removed. If you have reestablished the connection with the board, you have the liberty to choose the desired serial port.



Step 8: Submit the software.

Prior to addressing the process of uploading our software to the board, it is essential to demonstrate the functionality of each icon found on the Arduino IDE toolbar.



A-It verifies whether there were any errors throughout the compilation process.

B-A software application for manipulating the Arduino microcontroller board with programming instructions.

Pressing the C key on a keyboard initiates the creation of a new drawing.

To load any of the sample drawings, just double-click hereto save your drawing.

The F-A serial monitor, denoted by the initial "F," is responsible for both receiving and transmitting the serial data of the board.

Whenever you feel prepared, you can press the "Upload" button. After a brief period of time, the RX as well as Texas LEDs on the board will start to flash. After the upload process is finished, the status bar will display the message "Done uploading".

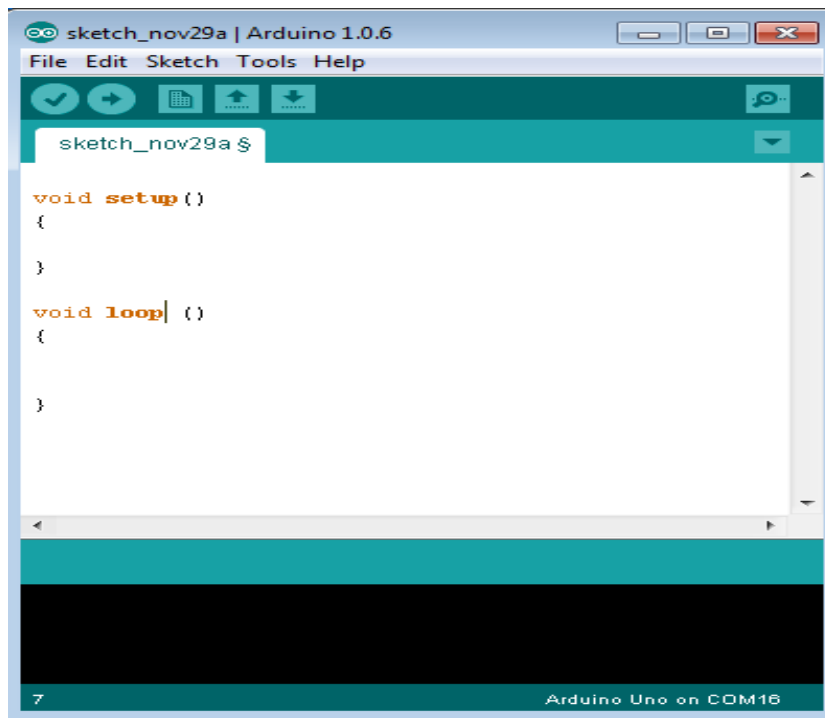
It is essential to press the reset button on the board in order to establish a connection between an NG Arduino Mini or any other board and the Arduino software.

ARDUINO CODE STRUCTURE

Learn the ins and outs of the Arduino language of programming and its associated terminology in this comprehensive chapter. Arduino is an open-source hardware platform. The C/C++ pic libraries and the Java runtime environment's source code are both made freely accessible under licenses. An Arduino programming tool is a new technological tool, and the name "sketch" characterizes it. An Arduino program primarily consists of three parts: structure, contents (constants or variables), and functions. If you want to write code for the Arduino platform that doesn't crash during compilation or have syntax errors, this is the book for you.

The Structure is where we should begin. There are primarily two purposes to software architecture: The function Setup

Iterative procedure



```
Void setup ( )  
{  
  
}
```

PURPOSE:

Transformers are heart of power production and distribution. Damages of transformer effect on power distribution. Damage happen due to poor maintenance and overheating. Sometimes over load create very huge problem. But there no transformer health monitoring system available in existing electricity department. Here we have solution like IOT based transformer health monitoring system.

INPUT

OUTPUT

RETURN

```
Void Loop ( )
```

```
{  
}
```

PURPOSE:

You may make your software adapt to new situations by using the loop () function once you've established a setup () function to initialize and configure the initial parameters. Use it to control an Arduino board.

INPUT

OUTPUT

RETURN

WORKING:

Here we can connect bulb (load) to transformer. Current sensor reads current value of load. Also voltage sensor reads voltage. Here we have load variation circuit and it varies voltage and current of load. LM35 reads temperature of transformer and update on LCD. Temperature value, Voltage and Current values will update to IOT server through WIFI (Esp8266/IOT module). User can see data in IOT server from anywhere. We can see data in graphical format also.

CHAPTER 7

SAMPLE CODE

```
#define BLYNK_TEMPLATE_ID "TMPL3FzE3JlrI"
#define BLYNK_TEMPLATE_NAME "gas cylinder level and leakage detection"
#define BLYNK_AUTH_TOKEN "codqhCYvi0sqvpH82utuPuFERt0bs26S"
char auth[] = BLYNK_AUTH_TOKEN;
//#define BLYNK_PRINT Serial
#include <ESP8266WiFi.h>
#include <BlynkSimpleEsp8266.h>
//#include <Ethernet.h>
BlynkTimer timer;
// Your WiFi credentials.
char ssid[] = "ABCDE";
char pass[] = "123456789";
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0X27,16,2);
const int trigPin = 15;
const int echoPin = 12;
long duration;
int distance;
#define Gas_sensor A0
#define Buzzer 13

void setup()
{
  lcd.init();
  lcd.begin(16,2);
  lcd.backlight();
  Serial.begin(9600);
  Blynk.begin(auth, ssid, pass);
  pinMode(trigPin, OUTPUT); // Sets the trigPin as an Output
  pinMode(echoPin, INPUT); // Sets the echoPin as an Input
  pinMode(Buzzer, OUTPUT);
```



```
pinMode(Gas_sensor,INPUT);
digitalWrite(Buzzer,LOW);
lcd.clear();
lcd.setCursor(0,0);
lcd.print("IOT BASED GAS");
lcd.setCursor(0,1);
lcd.print("CYLINDER LEVEL");
delay(2000);
lcd.clear();
lcd.setCursor(0,0);
lcd.print("  AND  ");
lcd.setCursor(0,1);
lcd.print("GAS LEAKAGE");
delay(2000);
lcd.clear();
lcd.setCursor(0,0);
lcd.print(" DETECTION ");
lcd.setCursor(0,1);
lcd.print(" SYSTEM ");
delay(2000);
}
```

```
void loop()
```

```
{
```

```
int Gas_sensor_state = analogRead(Gas_sensor);
lcd.clear();
lcd.setCursor(0,0);
lcd.print("Gas_state:");
lcd.setCursor(0,1);
lcd.print(Gas_sensor_state);
Blynk.virtualWrite(V0, Gas_sensor_state);
delay(1000);
```

```

if(Gas_sensor_state <100)
{
    digitalWrite(Buzzer,HIGH);
    lcd.clear();
    lcd.setCursor(0,0);
    lcd.print("GAS DETECTED");
    delay(1000);
}
else
{
    digitalWrite(Buzzer,LOW);
}
digitalWrite(trigPin, LOW);
delayMicroseconds(2);// Sets the trigPin on HIGH state for 10 micro seconds
digitalWrite(trigPin, HIGH);
delayMicroseconds(10);
digitalWrite(trigPin, LOW); // Reads the echoPin, returns the sound wave travel time in
microseconds
duration = pulseIn(echoPin, HIGH); // Calculating the distance
distance= duration*0.034/2;// Prints the distance on the Serial Monitor
Serial.print("Distance: ");
Serial.println(distance);
lcd.clear();
lcd.setCursor(0,0);
lcd.print("WATER LEVEL:");
lcd.setCursor(0,1);
lcd.print(distance);
Blynk.virtualWrite(V1, distance);
delay(500);
if(distance > 25)
{
    digitalWrite(Buzzer,HIGH);
    lcd.clear();
    lcd.setCursor(0,0);

```

```
    lcd.print("GAS LEVEL");  
    lcd.setCursor(0,1);  
    lcd.print("DECREASED");  
    delay(1000);  
}  
else  
{  
    digitalWrite(Buzzer,LOW);  
}  
Blynk.run();  
timer.run();  
}
```

CHAPTER 8

CONCLUSION

The "IOT Based Gas Cylinder Level and Leakage Detection" system provides a modern and effective solution for ensuring safety in homes and industries where LPG cylinders are used. By integrating ultrasonic sensors to monitor gas levels and MQ series sensors to detect leakage, the system offers continuous, real-time surveillance of gas cylinder conditions. The use of Node MCU allows for seamless data transmission to cloud platforms, enabling remote monitoring through smartphones or computers. In the event of a leakage, instant alerts via buzzers and mobile notifications ensure timely user intervention. This project not only enhances safety but also promotes efficient usage of gas by preventing unexpected depletion, making it a reliable and smart solution for gas management in today's digital era.

REFERENCES

- [1] K. Ramesh, "IOT Based Smart Gas Cylinder Monitoring System," International Journal of Engineering Research & Technology (IJERT), 2021.
- [2] A. Sharma and R. Patel, "Real Time Gas Leakage and Level Monitoring Using IOT," International Research Journal of Engineering and Technology (IRJET), 2020.
- [3] S. Gupta, "Development of LPG Gas Leakage and Level Detection System," Journal of Emerging Technologies and Innovative Research, 2019.
- [4] P. Bansal, "Smart Gas Cylinder Using IOT for Home Safety," International Journal of Computer Applications, 2018.
- [5] R. Singh and V. Mehta, "IOT Based Gas Cylinder Safety Alert System," International Journal of Scientific Research in Science and Technology (IJSRST), 2021.
- [6] N. Jadhav, "Automatic Gas Detection and Alerting System Using Arduino and IOT," International Journal of Engineering Sciences & Research Technology (IJESRT), 2019.
- [7] M. Khan, "Wireless LPG Monitoring and Alerting System Using ESP8266," Journal of Innovations in Engineering and Technology, 2020.
- [8] T. Das and A. Roy, "Gas Cylinder Weight and Leakage Detection System Using IOT," International Journal for Research in Applied Science and Engineering Technology (IJRASET), 2022.
- [9] S. Kulkarni, "Design of IOT-Based LPG Monitoring System," Journal of Industrial Safety Engineering, 2021.

- [10] V. Rathi and K. Jain, "Sensor Based Gas Detection and Level Indicator System," International Journal of Scientific & Engineering Research, 2018.
- [11] M. Verma, "Low Cost IOT Based Gas Level Monitoring System," International Conference on Electronics and Sustainable Communication Systems, 2020.
- [12] P. Raj, "Smart Gas Leakage Detection with Alerting and Auto Shut-Off," Journal of Embedded Systems and Applications, 2019.
- [13] A. Kale, "IOT Based Smart Home Gas Monitoring Using Node MCU," International Journal of Computer Science and Mobile Computing (IJCSMC), 2021.
- [14] R. Sinha and A. Mondal, "Leakage and Weight Detection System for LPG Cylinders," International Journal of Innovative Research in Technology (IJIRT), 2020.
- [15] L. Kumar, "An Arduino Based Gas Detection and Cylinder Monitoring System," Journal of Engineering and Technology, 2022.